

UPC

UNIVERSITAT POLITÈCNICA DE CATALUNYA
FACULTAT D'INFORMÀTICA DE BARCELONA

Grado en Ingeniería Informática

SemanticFIB

Buscador Semántico Inteligente
para la Facultad de Informática de Barcelona

Basado en Ontologías y Modelos de Lenguaje

Trabajo de Fin de Grado

Autor: Giancarlo Morales Muñoz

Director: Ramon Sangüesa

Departamento: Ingeniería de Servicios y Sistemas
de Información (ESSI)

Especialidad: Ingeniería del Software

Cuatrimestre de Otoño 2024–2025

Barcelona, 10 de junio de 2026

Resumen

Este Trabajo de Fin de Grado presenta el diseño, implementación y evaluación de **SemanticFIB**, un buscador semántico inteligente para la Facultad de Informática de Barcelona (FIB) de la UPC. El sistema combina técnicas de *Retrieval Augmented Generation* (RAG) con una ontología de dominio formal (RDF/OWL) para ofrecer respuestas fundamentadas a las consultas de los estudiantes sobre trámites, asignaturas, normativa y servicios de la facultad.

El trabajo sigue una metodología experimental comparativa: se implementan primero dos sistemas de referencia (*baselines*) —búsqueda léxica BM25 y búsqueda vectorial pura— y, sobre ellos, se construyen y evalúan tres estrategias progresivamente más sofisticadas: expansión ontológica de consultas, búsqueda guiada por ontología con vinculación de entidades y re-ranqueo controlado, y una fusión híbrida mediante *Reciprocal Rank Fusion* (RRF). Todas las estrategias se evalúan de forma automática y reproducible sobre un banco de 40 consultas reales con métricas estándar de recuperación de información (Hit@k, MRR, nDCG).

Los resultados muestran que la expansión de consultas por sí sola apenas mejora la búsqueda vectorial, mientras que la estrategia controlada —que explota el conocimiento estructurado de la ontología para inyectar páginas canónicas y re-ranquear candidatos— alcanza un **92,5 % de Hit@5 y un MRR de 0,82**, multiplicando por más de dos los resultados de los *baselines* y generalizando correctamente a consultas no vistas durante el diseño.

El resultado es un sistema funcional, desplegable localmente sin dependencias externas de pago, que proporciona respuestas precisas con citación de las fuentes originales.

Palabras clave: búsqueda semántica, ontologías, RDF, OWL, SPARQL, RAG, modelos de lenguaje, recuperación de información, BM25, ChromaDB, web scraping, FIB, UPC.

Abstract

This Bachelor’s Thesis presents the design, implementation, and evaluation of **SemanticFIB**, an intelligent semantic search engine for the Barcelona School of Informatics (FIB) at UPC. The system combines Retrieval Augmented Generation (RAG) techniques with a formal domain ontology (RDF/OWL) to provide grounded answers to student queries about procedures, courses, regulations, and faculty services.

The work follows a comparative experimental methodology: two baselines are implemented first —BM25 lexical search and pure vector search— and three increasingly sophisticated strategies are built and evaluated on top of them: ontology-based query expansion, ontology-guided retrieval with entity linking and controlled reranking, and a hybrid fusion via Reciprocal Rank Fusion (RRF). All strategies are evaluated automatically and reproducibly on a benchmark of 40 real queries using standard information retrieval metrics (Hit@k, MRR, nDCG).

Results show that query expansion alone barely improves vector search, while the controlled strategy —which exploits the structured knowledge of the ontology to inject canonical pages and rerank candidates— achieves **92.5 % Hit@5 and 0.82 MRR**, more than doubling baseline performance and generalizing correctly to queries unseen during design.

Keywords: semantic search, ontologies, RDF, OWL, SPARQL, RAG, language models, information retrieval, BM25, ChromaDB, web scraping, FIB, UPC.

Agradecimientos

Quiero agradecer a mi director de TFG, Ramon Sangüesa, por su orientación y apoyo durante todo el proyecto. También a la Facultad de Informática de Barcelona por proporcionar el entorno académico y los recursos necesarios para el desarrollo de este trabajo, y a los compañeros que dedicaron parte de su tiempo a probar el sistema y darme su opinión sincera.

Índice general

Resumen	I
Abstract	II
Agradecimientos	III
Lista de Acrónimos	IX
1. Introducción	1
1.1. Contexto	1
1.2. Motivación	1
1.3. Hipótesis y preguntas de investigación	2
1.4. Objetivos del proyecto	2
1.5. Alcance	3
1.6. Estructura del documento	3
2. Fundamentos Teóricos	4
2.1. Ontologías	4
2.1.1. Definición y propósito	4
2.1.2. RDF, OWL y SPARQL	4
2.1.3. Ontologías ligeras vs. formales	5
2.1.4. Aplicación en búsqueda semántica	5
2.2. Modelos de Lenguaje (LLMs)	5
2.2.1. Arquitectura Transformer	5
2.2.2. Modelos locales vs. API	6
2.3. Búsqueda Léxica: BM25	6
2.4. Búsqueda Semántica y Embeddings	6
2.4.1. Embeddings de texto	6
2.4.2. Similitud coseno	7
2.4.3. Modelo de embeddings multilingüe	7
2.5. Retrieval Augmented Generation (RAG)	7
2.5.1. Problema de las alucinaciones	7
2.5.2. Arquitectura RAG	8
2.5.3. Ventajas de RAG	8
2.6. Fusión de rankings: Reciprocal Rank Fusion	9
3. Estado del Arte	10
3.1. Sistemas de Q&A académicos	10
3.2. RAG aumentado con conocimiento estructurado	10

3.3. Posicionamiento de SemanticFIB	11
4. Identificación del Problema	12
4.1. Problemática actual	12
4.2. Actores implicados	13
4.3. Solución propuesta	13
5. Alcance del Proyecto	14
5.1. Objetivos específicos	14
5.2. Requisitos funcionales	15
5.3. Requisitos no funcionales	15
5.4. Obstáculos y riesgos	16
6. Metodología y Herramientas	17
6.1. Metodología de desarrollo	17
6.2. Herramientas y tecnologías	18
6.3. Entorno de desarrollo	18
7. Arquitectura del Sistema	19
7.1. Visión general	19
7.2. Estructura de ficheros	20
7.3. Flujo de datos	21
7.3.1. Pipeline Offline (Indexación)	21
7.3.2. Pipeline Online (Consulta)	21
8. Implementación	23
8.1. Adquisición de datos: Web Scraping	23
8.1.1. Punto de partida	23
8.1.2. Problemas encontrados	24
8.2. La ontología de dominio	24
8.2.1. Primera iteración: un diccionario de sinónimos	24
8.2.2. Segunda iteración: grafo formal RDF/OWL	24
8.2.3. Población semiautomática	26
8.2.4. API de consulta	27
8.3. Indexación: del HTML a ChromaDB	28
8.3.1. Deduplicación con prioridad canónica	28
8.3.2. Chunking semántico	29
8.3.3. Filtrado de fragmentos de bajo valor y páginas de aterrizaje	29
8.3.4. Identificadores robustos e idempotencia	29
8.4. Las cinco estrategias de recuperación	29
8.4.1. Baseline 1: BM25 (léxico)	30
8.4.2. Baseline 2: búsqueda vectorial pura (dense)	30
8.4.3. Estrategia 3: expansión ontológica naive	30
8.4.4. Vinculación de entidades (Entity Linking)	30
8.4.5. Estrategia 4: búsqueda controlada por ontología	31
8.4.6. Estrategia 5: fusión híbrida (RRF)	32
8.5. Pipeline RAG y generación	32
8.5.1. Condensación de preguntas de seguimiento	32
8.5.2. Prompt y fundamentación	32

8.6. Interfaz de usuario	32
9. Evaluación y Resultados	34
9.1. Diseño de la evaluación	34
9.1.1. Del análisis cualitativo a la evaluación formal	34
9.1.2. El banco de consultas (golden set)	34
9.1.3. Métricas	35
9.2. Resultados de la recuperación	35
9.2.1. Generalización: dev vs. test	36
9.3. Impacto de la ontología: análisis por mecanismo	37
9.4. Evaluación de la estrategia híbrida (RRF)	38
9.5. Evaluación cualitativa del sistema completo	38
9.6. Evaluación con usuarios (SUS): diseño del estudio	39
9.6.1. Metodología diseñada	39
9.7. Limitaciones observadas	39
10. Planificación Temporal y Económica	41
10.1. Planificación temporal	41
10.2. Gestión económica	42
10.2.1. Costes de personal	42
10.2.2. Costes de hardware y software	42
10.2.3. Presupuesto total	42
11. Sostenibilidad	43
11.1. Dimensión económica	43
11.2. Dimensión ambiental	43
11.3. Dimensión social	43
12. Conclusiones y Trabajo Futuro	45
12.1. Conclusiones	45
12.2. Objetivos alcanzados	46
12.3. Trabajo futuro	46
A. Instrucciones de Instalación	49
A.1. Requisitos previos	49
A.2. Instalación y ejecución	49
B. Extracto de la Ontología (Turtle)	51
C. Ejemplos de Consultas	52
D. Categorías del Banco de Consultas	53

Índice de figuras

2.1. Arquitectura RAG básica con enriquecimiento ontológico. Esta fue la primera versión del sistema; la versión final (Figura 7.1) sustituye la búsqueda vectorial simple por un módulo de recuperación con cinco estrategias conmutables.	8
6.1. Sprints del proyecto	17
7.1. Arquitectura completa del sistema SemanticFIB. La ontología participa en ambos pipelines: en la ingesta (preservación de páginas canónicas) y en la consulta (expansión, entity linking, re-ranqueo).	20
8.1. Arquitectura de la ontología en dos capas: distinción formal entre el esquema conceptual (TBox) y la población de instancias (ABox). . . .	25
8.2. Pipeline de población semiautomática: la ontología se reconstruye desde los datos reales de la facultad, de modo que un cambio en la web se incorpora re-ejecutando el constructor.	26
8.3. Visualización del grafo RDF de la ontología de la FIB. En el centro, las clases del esquema; alrededor, las instancias de asignaturas, titulaciones y conceptos con sus relaciones.	27
9.1. Hit@1 y Hit@5 de las cinco estrategias sobre el conjunto completo de 40 consultas.	36

Índice de cuadros

2.1. Comparativa modelos locales vs. API	6
3.1. Comparativa de frameworks y herramientas relacionadas	11
4.1. Actores del sistema y sus intereses	13
5.1. Requisitos funcionales del sistema	15
5.2. Requisitos no funcionales	15
5.3. Riesgos identificados y mitigaciones	16
6.1. Stack tecnológico del proyecto	18
8.1. Propiedades de datos y su rol operativo en el <i>retrieval</i>	26
9.1. Resultados globales de las cinco estrategias de recuperación (40 consultas)	35
9.2. Desglose por partición: 20 consultas de desarrollo vs. 20 consultas held-out	37
10.1. Desglose temporal por sprints	41
10.2. Costes de personal	42
10.3. Costes genéricos (hardware y software)	42
10.4. Presupuesto total del proyecto	42
12.1. Grado de cumplimiento de los objetivos	46
C.1. Ejemplos de consultas y comportamiento del sistema (estrategia controlada)	52
D.1. Distribución del golden set por categoría (40 consultas, 20 dev / 20 test; 37 en catalán, 3 en castellano)	53

Lista de Acrónimos

ABox	Assertional Box (instancias de una ontología)
API	Application Programming Interface
BM25	Best Matching 25 (función de ranking léxico)
CSS	Cascading Style Sheets
FIB	Facultat d'Informàtica de Barcelona
GEI	Grau en Enginyeria Informàtica
HTML	HyperText Markup Language
IR	Information Retrieval
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	Normalized Discounted Cumulative Gain
NLP	Natural Language Processing
OWL	Web Ontology Language
PDF	Portable Document Format
RAG	Retrieval Augmented Generation
RDF	Resource Description Framework
RRF	Reciprocal Rank Fusion
SKOS	Simple Knowledge Organization System
SPARQL	SPARQL Protocol and RDF Query Language
SUS	System Usability Scale
TBox	Terminological Box (esquema de una ontología)
TFG	Trabajo de Fin de Grado
TTL	Turtle (serialización de RDF)
UPC	Universitat Politècnica de Catalunya
URI	Uniform Resource Identifier

Capítulo 1

Introducción

1.1. Contexto

La Facultat d’Informàtica de Barcelona (FIB) de la Universitat Politècnica de Catalunya (UPC) es uno de los centros de referencia en formación informática en Cataluña y España. Con más de 3.000 estudiantes activos, múltiples grados y másters, centenares de asignaturas y una variedad notable de trámites académicos, la cantidad de información disponible para los estudiantes es enorme y, sobre todo, dispersa.

Cualquier estudiante de la facultad ha vivido la situación: quieres saber si quedan plazas libres en una asignatura, o cuándo abre el período de matrícula, o qué prerequisites tiene PROP, y acabas saltando entre cinco o seis páginas de la web, el Racó, algún PDF de normativa y, si la cosa se complica, la cola de secretaría. La información existe —la web de la FIB es completa y se mantiene actualizada— pero el camino hasta ella no siempre es evidente, especialmente cuando uno no conoce el vocabulario “oficial” con el que está etiquetada.

Este desajuste entre cómo pregunta un estudiante (“*em vull apuntar a les assignatures*”) y cómo está organizada la información (“Matrícula > Estudiants de grau > Períodes”) es, en el fondo, un problema clásico de recuperación de información con vocabulario desalineado, y es el problema que este trabajo intenta resolver.

1.2. Motivación

Los avances recientes en modelos de lenguaje (LLMs) y búsqueda semántica ofrecen una oportunidad única para crear herramientas que permitan a los estudiantes obtener respuestas rápidas, precisas y fundamentadas. Sin embargo, el uso directo de un LLM presenta un problema fundamental: las **alucinaciones**. Un modelo genérico puede inventar información sobre la FIB que suene convincente pero sea incorrecta; en un contexto donde la respuesta puede afectar a la matrícula o a un trámite con plazos, esto es inaceptable.

La técnica de *Retrieval Augmented Generation* (RAG) mitiga este problema combinando la capacidad generativa de los LLMs con un sistema de recuperación de documentos que ancla las respuestas en información real y verificable. Pero el RAG tiene su propio talón de Aquiles, que descubrí muy pronto durante el desarrollo: **la calidad de la respuesta final depende casi por completo de la calidad de la recuperación**. Si los documentos que llegan al modelo no son los correctos, ninguna habilidad generativa puede salvar la respuesta.

Por eso este TFG no trata simplemente de “montar un chatbot RAG”, sino de estudiar de forma sistemática **cómo mejorar la fase de recuperación usando conocimiento estructurado del dominio** —una ontología— y de medir con rigor cuánto aporta cada mecanismo.

1.3. Hipótesis y preguntas de investigación

La hipótesis central del trabajo es que, en un dominio cerrado y bien estructurado como el de una facultad universitaria, **el conocimiento explícito del dominio (ontología) puede mejorar significativamente la recuperación semántica** respecto a los enfoques puramente estadísticos (embeddings) o léxicos (BM25).

De esta hipótesis se derivan tres preguntas concretas que la evaluación del Capítulo 9 intenta responder:

1. **P1:** ¿Mejora la recuperación el simple hecho de expandir la consulta con sinónimos y términos relacionados de la ontología?
2. **P2:** ¿Qué aporta una integración más profunda de la ontología (vinculación de entidades, páginas canónicas, re-ranqueo guiado) frente a la expansión simple?
3. **P3:** ¿Es preferible una fusión híbrida de señales (léxica + vectorial + ontológica) o una única estrategia dominante?

Adelanto aquí, porque condiciona la estructura de la memoria, que la respuesta a P1 resultó ser esencialmente negativa, y que ese resultado —inesperado para mí al principio— fue el que reorientó el diseño del sistema hacia la estrategia de re-ranqueo controlado que acabó siendo la contribución principal del trabajo.

1.4. Objetivos del proyecto

El objetivo principal de este TFG es diseñar, implementar y evaluar un buscador semántico inteligente para la FIB que:

1. Recupere información real y actualizada de la web de la FIB.
2. Modele el dominio académico mediante una ontología formal (RDF/OWL) poblada de forma semiautomática a partir de los propios datos de la web.
3. Implemente y compare múltiples estrategias de recuperación: *baselines* léxico y vectorial, expansión ontológica, búsqueda controlada por ontología y fusión híbrida.
4. Evalúe las estrategias de forma automática y reproducible con métricas estándar de IR sobre un banco de consultas reales.
5. Genere respuestas naturales basadas exclusivamente en documentos verificados, citando las fuentes originales.
6. Funcione localmente sin dependencias de servicios de pago externos.

1.5. Alcance

El proyecto cubre el ciclo completo: desde la adquisición de datos (*web scraping* de `fib.upc.edu`), pasando por el procesamiento e indexación (limpieza, deduplicación, *chunking*, *embeddings*, ChromaDB), la modelización del dominio (ontología RDF/OWL con población semiautomática), la implementación de cinco estrategias de recuperación comparables, la evaluación cuantitativa, y la interfaz de usuario (Streamlit) con generación de respuestas (LLM local).

Quedan fuera del alcance la integración con sistemas internos de la UPC (Racó, eSecretaria), el soporte de datos personalizados por estudiante, y el despliegue en producción con múltiples usuarios concurrentes; todos ellos se discuten como trabajo futuro.

1.6. Estructura del documento

La memoria se organiza de la siguiente manera:

- **Capítulo 2:** Conceptos clave y fundamentos teóricos: ontologías, embeddings, BM25, RAG y fusión de rankings.
- **Capítulo 3:** Estado del arte en buscadores semánticos, RAG y RAG aumentado con grafos de conocimiento.
- **Capítulo 4:** Identificación del problema y actores implicados.
- **Capítulo 5:** Alcance, requisitos y riesgos.
- **Capítulo 6:** Metodología y herramientas.
- **Capítulo 7:** Arquitectura del sistema.
- **Capítulo 8:** Implementación: el grueso del trabajo, contado en el orden en que se desarrolló.
- **Capítulo 9:** Evaluación y resultados.
- **Capítulo 10:** Planificación temporal y económica.
- **Capítulo 11:** Sostenibilidad.
- **Capítulo 12:** Conclusiones y trabajo futuro.

Capítulo 2

Fundamentos Teóricos

2.1. Ontologías

2.1.1. Definición y propósito

Una ontología, en el contexto de la informática, es una representación formal y explícita de un conjunto de conceptos dentro de un dominio y las relaciones entre ellos [1]. A diferencia de un simple glosario o taxonomía, una ontología define:

- **Conceptos** (clases): entidades del dominio (ej. *Asignatura*, *Titulación*).
- **Propiedades**: atributos de los conceptos (ej. *código*, *recurso canónico*).
- **Relaciones**: conexiones semánticas entre conceptos (ej. *pertenece_a*, *tiene_especialidad*).
- **Instancias**: individuos concretos de las clases (ej. la asignatura XC, el grado GEI).
- **Axiomas**: reglas lógicas que restringen el modelo.

Una distinción que resulta central en este trabajo es la separación entre el **TBox** (*Terminological Box*), que contiene el esquema conceptual —las clases y propiedades—, y el **ABox** (*Assertional Box*), que contiene las afirmaciones sobre instancias concretas. Esta separación es precisamente lo que distingue una ontología de un simple diccionario de sinónimos: el esquema impone una estructura sobre la que las instancias se pueblan, se validan y se consultan.

2.1.2. RDF, OWL y SPARQL

El estándar de facto para representar ontologías en la web semántica es la pila de tecnologías del W3C:

- **RDF** (*Resource Description Framework*): modelo de datos basado en tripletas (*sujeto*, *predicado*, *objeto*). Todo el conocimiento se expresa como un grafo dirigido etiquetado.
- **RDFS y OWL** (*Web Ontology Language*): vocabularios que añaden semántica al grafo: jerarquías de clases (`rdfs:subClassOf`), tipos de propiedades (`owl:ObjectProperty`, `owl:DatatypeProperty`), dominios y rangos.

- **SKOS** (*Simple Knowledge Organization System*): vocabulario para sistemas de organización del conocimiento; en este trabajo se usa para las etiquetas multilingües (`skos:prefLabel`, `skos:altLabel`) y las relaciones asociativas (`skos:related`).
- **SPARQL**: lenguaje de consulta sobre grafos RDF, análogo a SQL para bases de datos relacionales. Permite recuperar subgrafos mediante patrones de tripletas.
- **Turtle (TTL)**: serialización textual legible de RDF, que es el formato en que se persiste la ontología de este proyecto.

2.1.3. Ontologías ligeras vs. formales

Para aplicaciones de búsqueda semántica es común utilizar ontologías ligeras que capturan vocabulario y relaciones sin la complejidad de una lógica descriptiva completa (como OWL-DL con razonadores). En este trabajo se adopta un punto intermedio que me parece honesto explicitar: la ontología es **formal en su representación** (RDF/OWL, consultable con SPARQL, inspeccionable con Protégé) pero **ligera en su axiomatización** (no se utiliza razonamiento lógico automático, sino consultas de grafo). La justificación es práctica: para el caso de uso —enriquecer y guiar la recuperación— la navegación del grafo es suficiente, y el coste de un razonador no aporta beneficio medible.

2.1.4. Aplicación en búsqueda semántica

El uso de una ontología de dominio en un sistema de búsqueda permite, al menos en teoría:

1. **Expansión de consulta:** añadir sinónimos y términos relacionados.
2. **Desambiguación:** identificar el sentido correcto de un término.
3. **Vinculación de entidades:** reconocer menciones de entidades concretas (códigos de asignatura, siglas de titulaciones) y resolverlas a sus recursos.
4. **Contexto semántico:** proporcionar información estructural al LLM.
5. **Conocimiento canónico:** saber cuál es *la* página oficial de cada concepto, independientemente de su contenido textual.

Digo “en teoría” porque una parte importante de este TFG consiste justamente en medir cuáles de estos mecanismos aportan mejora real y cuáles no. Como se verá en el Capítulo 9, los resultados son menos intuitivos de lo que esperaba al empezar.

2.2. Modelos de Lenguaje (LLMs)

2.2.1. Arquitectura Transformer

Los modelos de lenguaje actuales se basan en la arquitectura Transformer [2], que utiliza mecanismos de atención para procesar secuencias de texto de manera paralela. Esto permite capturar dependencias a larga distancia en el texto y entrenar modelos sobre corpus masivos, dando lugar a los *Large Language Models* capaces de generar texto coherente y responder preguntas en lenguaje natural.

2.2.2. Modelos locales vs. API

Existen dos aproximaciones principales para utilizar LLMs en una aplicación:

Cuadro 2.1: Comparativa modelos locales vs. API

Aspecto	Modelo Local (Ollama)	API (OpenAI)
Coste	Gratuito (hardware propio)	Por token (\$0.002–\$0.06/1K)
Privacidad	Total (datos locales)	Datos enviados a terceros
Latencia	Variable (depende del hardware)	Consistente (<2s)
Calidad	Buena (Llama 3.2 3B)	Excelente (GPT-4o)
Disponibilidad	Offline posible	Requiere Internet

En este proyecto se utiliza por defecto Ollama con **Llama 3.2 (3B parámetros)** para garantizar independencia de servicios externos y coste cero, aunque el sistema permite conmutar a la API de OpenAI mediante configuración. Cabe señalar que el papel del LLM en la arquitectura final es deliberadamente acotado: genera la respuesta a partir de los documentos recuperados y condensa preguntas de seguimiento, pero **no** interviene en el ranking de documentos, que es donde se concentra la contribución del trabajo.

2.3. Búsqueda Léxica: BM25

Antes de la era de los embeddings, la recuperación de información se basaba en la coincidencia léxica ponderada. **Okapi BM25** [5] sigue siendo, décadas después, un *baseline* extraordinariamente competitivo y es la función de ranking de motores como Elasticsearch o Lucene. Para una consulta Q con términos $q_1, \dots, q_n$ y un documento D :

$$\text{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (2.1)$$

donde $f(q_i, D)$ es la frecuencia del término en el documento, $|D|$ la longitud del documento, avgdl la longitud media del corpus, y k_1 , b parámetros de saturación y normalización (valores estándar $k_1 = 1.5$, $b = 0.75$).

La intuición es simple: un documento es relevante si contiene los términos de la consulta, con más peso para términos raros (IDF alto) y con saturación para evitar que la repetición artificiosa de un término dispare el score. BM25 es **fuerte en coincidencias exactas** (códigos, nombres propios) y **ciego a la sinonimia**: “profesor” y “docent” son términos distintos para él. Este perfil complementario al de los embeddings es lo que motiva los sistemas híbridos.

2.4. Búsqueda Semántica y Embeddings

2.4.1. Embeddings de texto

Un *embedding* es una representación numérica (vector) de un texto en un espacio multidimensional donde la proximidad geométrica refleja la similitud semántica.

Formalmente:

$$f : \text{Texto} \rightarrow \mathbb{R}^d \quad (2.2)$$

donde d es la dimensionalidad del espacio (384 en nuestro caso con MiniLM).

2.4.2. Similitud coseno

La similitud entre dos vectores de embedding se calcula mediante la similitud coseno:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|} = \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2} \cdot \sqrt{\sum_{i=1}^d b_i^2}} \quad (2.3)$$

El rango es $[-1, 1]$, donde 1 indica vectores idénticos y 0 indica ortogonalidad (ninguna relación semántica).

2.4.3. Modelo de embeddings multilingüe

Se utiliza el modelo `paraphrase-multilingual-MiniLM-L12-v2` de Sentence Transformers [4], que:

- Soporta 50+ idiomas (incluidos catalán y castellano), algo imprescindible dado que el corpus está en catalán pero los estudiantes preguntan indistintamente en catalán y castellano.
- Genera vectores de 384 dimensiones.
- Es ligero ($\sim 120\text{MB}$) y rápido de ejecutar en CPU, lo que permite indexar y consultar en un portátil sin GPU dedicada.
- Ofrece buena calidad en tareas de similitud semántica.

La contrapartida de los embeddings, que sufrimos repetidamente durante el desarrollo, es que son **débiles ante tokens fuera de vocabulario semántico**: un código de asignatura como “XC” o “PROP” apenas aporta señal al vector, porque el modelo no tiene forma de saber que se refiere a una asignatura concreta de la FIB. Esta limitación motiva el módulo de vinculación de entidades del Capítulo 8.

2.5. Retrieval Augmented Generation (RAG)

2.5.1. Problema de las alucinaciones

Los LLMs generativos tienen tendencia a producir información plausible pero incorrecta (alucinaciones). En un contexto académico donde la precisión es crítica —plazos de matrícula, requisitos de normativa— esto es inaceptable.

2.5.2. Arquitectura RAG

RAG [3] combina recuperación de documentos con generación de texto:

1. **Retrieval:** dada una consulta, se recuperan los k documentos más relevantes de una base de datos.
2. **Augmentation:** los documentos recuperados se añaden al contexto del LLM.
3. **Generation:** el LLM genera una respuesta basada exclusivamente en el contexto proporcionado.

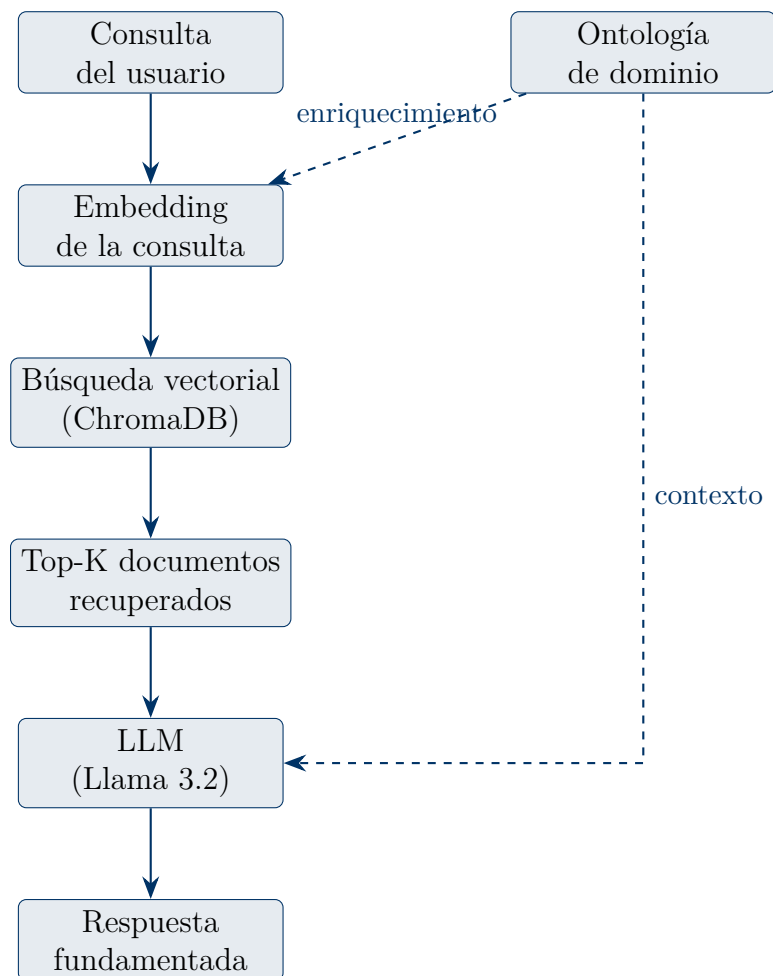


Figura 2.1: Arquitectura RAG básica con enriquecimiento ontológico. Esta fue la primera versión del sistema; la versión final (Figura 7.1) sustituye la búsqueda vectorial simple por un módulo de recuperación con cinco estrategias conmutables.

2.5.3. Ventajas de RAG

- **Fundamentación:** las respuestas se basan en documentos reales.
- **Citabilidad:** se puede indicar la fuente de cada afirmación.
- **Actualización:** no es necesario reentrenar el modelo para añadir información nueva; basta re-indexar.
- **Transparencia:** el usuario puede verificar las fuentes citadas.

2.6. Fusión de rankings: Reciprocal Rank Fusion

Cuando se dispone de varios sistemas de recuperación con perfiles complementarios (léxico, vectorial, ontológico), una pregunta natural es cómo combinarlos. *Reciprocal Rank Fusion* (RRF) [6] es un método simple y sorprendentemente robusto que fusiona listas rankeadas sin necesidad de calibrar los scores de cada sistema (que viven en escalas incomparables):

$$\text{RRF}(d) = \sum_{s \in S} \frac{w_s}{k + \text{rank}_s(d)} \quad (2.4)$$

donde S es el conjunto de sistemas, $\text{rank}_s(d)$ la posición del documento d en el ranking del sistema s , w_s un peso opcional por sistema y k una constante de suavizado (el valor $k = 60$ de la literatura original). La gracia de RRF es que solo usa posiciones, no scores, evitando el problema de normalizar magnitudes heterogéneas. Su limitación —que comprobamos empíricamente en la Sección 9.4— es que también *diluye* las señales de alta precisión cuando uno de los sistemas es claramente superior a los demás.

Capítulo 3

Estado del Arte

3.1. Sistemas de Q&A académicos

Diversas universidades han explorado chatbots y asistentes basados en IA para atender consultas de estudiantes. La mayoría utilizan aproximaciones basadas en reglas o *intents* predefinidos (tipo Dialogflow), con limitaciones evidentes en la cobertura y naturalidad de las respuestas: cada pregunta nueva exige añadir un *intent* a mano, y cualquier formulación fuera del guion produce respuestas genéricas.

Los sistemas más recientes incorporan modelos de lenguaje, pero a menudo sin mecanismos de fundamentación (*grounding*) que garanticen la precisión en el dominio específico, lo que los hace vulnerables a alucinaciones sobre normativas o plazos concretos.

3.2. RAG aumentado con conocimiento estructurado

La idea de combinar recuperación neuronal con conocimiento estructurado ha ganado tracción en los últimos años y conviene situar este TFG dentro de esa línea:

- **GraphRAG** [7] (Microsoft) construye automáticamente un grafo de conocimiento a partir del corpus mediante LLMs y lo usa para responder preguntas globales que requieren agregación. Es la referencia más visible del paradigma, aunque su coste de construcción (múltiples pasadas de LLM sobre todo el corpus) es elevado.
- **KAPING** [8] inyecta tripletas de un grafo de conocimiento directamente en el prompt del LLM, sin reentrenamiento, mejorando la factualidad en preguntas sobre entidades.
- **Query expansion clásica con ontologías** [9]: la expansión de consultas con tesauros y ontologías es una técnica veterana de IR, con resultados históricamente mixtos: funciona cuando la expansión es precisa y perjudica cuando introduce ruido —un antecedente directo del resultado negativo que obtenemos en P1.

3.3. Posicionamiento de SemanticFIB

Frente a estos trabajos, SemanticFIB se posiciona en un nicho concreto y deliberadamente acotado:

1. **Ontología de dominio específica y curada**, poblada semiautomáticamente desde la web institucional, en lugar de un grafo extraído automáticamente por LLMs (más barato, más controlable, y adecuado para un dominio pequeño y estable).
2. **La ontología actúa en la fase de recuperación** (expansión, vinculación de entidades, re-ranqueo), no solo como contexto del prompt.
3. **Evaluación comparativa sistemática** de cinco estrategias sobre el mismo corpus y el mismo banco de consultas, con división dev/test para controlar el sobreajuste de las reglas.
4. **100 % local y reproducible**: sin suscripciones, sin enviar datos a terceros.
5. **Dominio académico catalán/castellano**, con los retos multilingües que implica.

Cuadro 3.1: Comparativa de frameworks y herramientas relacionadas

Herramienta	Descripción	Local	Ontología	Gratuito
LangChain	Framework orquestación LLM	Parcial	No	Sí
LlamaIndex	Indexación y RAG	Parcial	Parcial	Sí
Haystack	Pipeline NLP completo	Parcial	No	Sí
GraphRAG	Grafo autoconstruido + RAG	Parcial	Automática	Sí
SemanticFIB	RAG + ontología de dominio formal	Sí	Sí (RDF/OWL)	Sí

Capítulo 4

Identificación del Problema

4.1. Problemática actual

Los estudiantes de la FIB se enfrentan a diversas dificultades para acceder a información académica:

- Información dispersa en múltiples secciones de la web.
- Navegación compleja con estructuras profundas (hasta 5–6 niveles de menú).
- Vocabulario técnico-administrativo que no coincide con el vocabulario del estudiante.
- Páginas críticas (horarios, exámenes, plazas libres) con muy poco texto estático, difíciles de encontrar por búsqueda de contenido.
- Horarios limitados de atención presencial en secretaría.
- Falta de un punto único de acceso a toda la información.

Vale la pena detenerse en el cuarto punto, porque no era obvio al inicio del proyecto y acabó condicionando el diseño: muchas de las páginas que un estudiante más necesita (los horarios, el calendario de exámenes, las plazas libres) son páginas cuyo contenido útil se genera dinámicamente o se presenta en tablas, de modo que su texto indexable es mínimo. Cualquier sistema basado puramente en similitud de contenido las penaliza, justo al revés de lo que el usuario espera.

4.2. Actores implicados

Cuadro 4.1: Actores del sistema y sus intereses

Actor	Interés
Estudiantes de grado	Acceder rápidamente a información sobre asignaturas, matrícula, trámites
Estudiantes de máster	Consultar normativa específica, movilidad, TFM
Futuros estudiantes	Notas de corte, planes de estudios, proceso de acceso
Personal de secretaría	Reducir consultas repetitivas, liberar tiempo
Profesorado	Derivar alumnos a información oficial
Equipo directivo	Mejorar la satisfacción de los estudiantes

4.3. Solución propuesta

SemanticFIB aborda este problema proporcionando:

- Un punto único de acceso conversacional.
- Comprensión semántica de las consultas (no solo palabras clave), reforzada con conocimiento explícito del dominio.
- Respuestas fundamentadas con fuentes verificables.
- Tratamiento específico de las páginas canónicas de bajo contenido textual.
- Disponibilidad 24/7 sin coste operativo.

Capítulo 5

Alcance del Proyecto

5.1. Objetivos específicos

1. Implementar un scraper robusto para `fib.upc.edu` que extraiga información estructurada.
2. Diseñar una ontología formal (RDF/OWL) del dominio académico de la FIB, poblada semiautomáticamente.
3. Implementar cinco estrategias de recuperación comparables bajo una interfaz común.
4. Construir un pipeline RAG que integre ontología, embeddings y LLM.
5. Crear un banco de consultas etiquetado y un sistema de evaluación automática con métricas de IR.
6. Desarrollar una interfaz web usable que permita comparar las estrategias en vivo.

5.2. Requisitos funcionales

Cuadro 5.1: Requisitos funcionales del sistema

ID	Prioridad	Descripción
RF01	Alta	El sistema debe responder preguntas en lenguaje natural (catalán y castellano)
RF02	Alta	Las respuestas deben citar fuentes originales (URLs)
RF03	Alta	El sistema debe funcionar sin conexión a APIs de pago
RF04	Alta	El sistema debe ofrecer múltiples estrategias de recuperación conmutables
RF05	Alta	La evaluación de las estrategias debe ser automática y reproducible
RF06	Media	La ontología debe enriquecer y guiar la recuperación
RF07	Media	La interfaz debe mostrar detalles de la búsqueda (documentos, scores, boosts)
RF08	Media	Mantener historial de conversación con condensación de preguntas de seguimiento
RF09	Baja	Soportar indexación de documentos PDF adicionales

5.3. Requisitos no funcionales

Cuadro 5.2: Requisitos no funcionales

ID	Prioridad	Descripción
RNF01	Alta	Tiempo de respuesta de recuperación inferior a 2 segundos
RNF02	Alta	Despliegue local sin infraestructura externa
RNF03	Media	Interfaz responsive y profesional
RNF04	Media	Código modular y extensible (añadir una estrategia no debe tocar las demás)
RNF05	Media	Ampliar la cobertura de la ontología sin modificar código
RNF06	Baja	Documentación completa del sistema

5.4. Obstáculos y riesgos

Cuadro 5.3: Riesgos identificados y mitigaciones

Riesgo	Prob.	Impacto	Mitigación
Cambios en web FIB	Media	Alto	Selectores flexibles, múltiples fallbacks
Calidad LLM local	Baja	Medio	Prompt engineering, restricción a docs
Sobreajuste de las reglas al banco de consultas	Media	Alto	División dev/test con consultas held-out
Datos duplicados / ruido del scraping	Alta	Medio	Deduplicación, filtros de calidad en ingesta
Embeddings no multilingües	Baja	Alto	Modelo MiniLM multilingüe (50+ idiomas)

El tercer riesgo merece comentario: cuando un sistema incorpora reglas diseñadas a partir del análisis de errores, existe el peligro de evaluar el sistema con las mismas consultas que motivaron las reglas, inflándose artificialmente los resultados. Fui consciente de este riesgo a mitad del desarrollo y la mitigación (separar consultas de diseño y consultas held-out) se explica en la Sección 9.1.2.

Capítulo 6

Metodología y Herramientas

6.1. Metodología de desarrollo

Se ha seguido una metodología **ágil iterativa** con sprints de 2-3 semanas. Cada sprint produce un incremento funcional del sistema que puede evaluarse de forma independiente. Más allá de la planificación, el desarrollo real siguió un ciclo que se repitió varias veces y que estructura el Capítulo 8: *implementar* → *evaluar* → *analizar errores* → *rediseñar*. Varias de las decisiones más importantes del proyecto (el re-ranqueo controlado, la vinculación de entidades, la deduplicación canónica) no estaban en el diseño inicial: surgieron del análisis de fallos concretos.

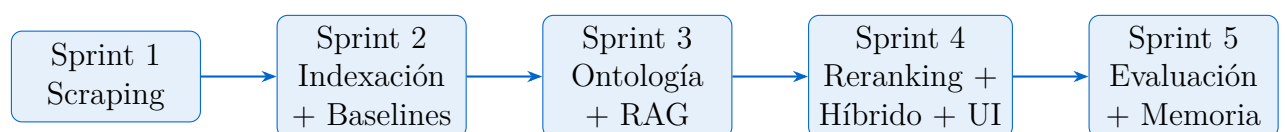


Figura 6.1: Sprints del proyecto

6.2. Herramientas y tecnologías

Cuadro 6.1: Stack tecnológico del proyecto

Componente	Tecnología	Justificación
Lenguaje	Python 3.9+	Ecosistema ML/NLP, productividad
LLM	Ollama + Llama 3.2 (3B)	Local, gratuito, buena calidad
Embeddings	Sentence Transformers	Multilingüe, ligero (120MB)
Vector DB	ChromaDB	Embebido, zero-config, persistente
Búsqueda léxica	rank-bm25	Implementación Okapi BM25 pura en Python
Ontología	rdflib (RDF/OWL/SPARQL)	Estándar W3C, serialización Turtle
Orquestación	LangChain	Abstracciones RAG maduras
Scraping	BeautifulSoup + requests	Robusto, flexible, ligero
Frontend	Streamlit	Prototipado rápido, componentes ricos
Control de versiones	Git	Estándar de la industria

La elección de `rdflib` frente a alternativas más pesadas (Apache Jena, GraphDB) responde al mismo criterio que el resto del stack: todo el sistema debe poder ejecutarse en un portátil sin servicios externos. `rdflib` carga el grafo en memoria (poco más de mil tripletas en nuestro caso), soporta SPARQL completo y serializa a Turtle, que es todo lo que el proyecto necesita.

6.3. Entorno de desarrollo

- **Sistema operativo:** macOS (Apple Silicon)
- **Editor:** Visual Studio Code con extensiones Python
- **Gestor de paquetes:** pip + entorno virtual (venv)
- **Inspección de la ontología:** Protégé / visualizadores RDF

Capítulo 7

Arquitectura del Sistema

7.1. Visión general

SemanticFIB sigue una arquitectura modular con separación clara de responsabilidades. El flujo de datos tiene dos caminos: **offline** (adquisición e indexación) y **online** (consulta). La diferencia más importante respecto a un RAG convencional es el módulo de recuperación: en lugar de una única búsqueda vectorial, existe una **factoría de estrategias** que expone cinco modos conmutables bajo la misma interfaz, lo que permite tanto la comparación experimental del Capítulo 9 como la selección en vivo desde la interfaz.

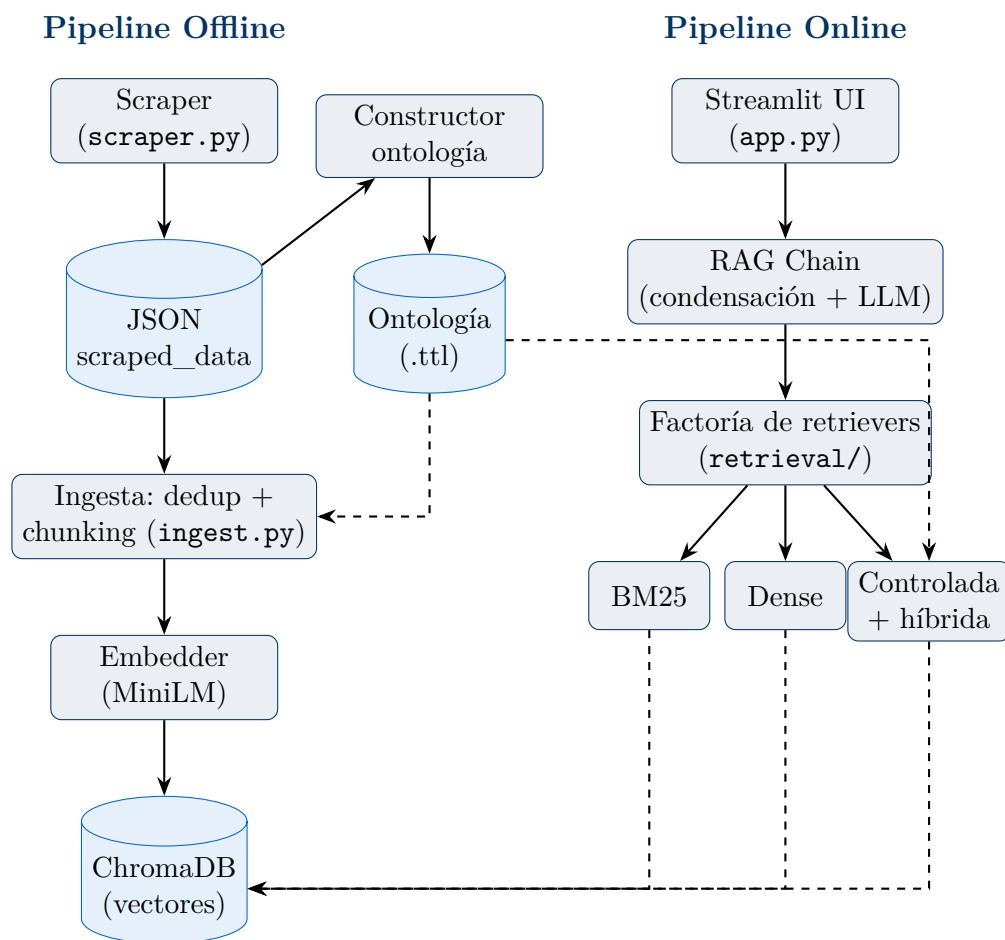


Figura 7.1: Arquitectura completa del sistema SemanticFIB. La ontología participa en ambos pipelines: en la ingestión (preservación de páginas canónicas) y en la consulta (expansión, entity linking, re-ranqueo).

7.2. Estructura de ficheros

Listing 7.1: Estructura del proyecto

```

1 semanticfib/
2 |-- app.py                # Interfaz Streamlit
3 |-- scraper.py            # Web scraper FIB
4 |-- ingest.py             # Indexación: dedup + chunking +
   embeddings
5 |-- settings.py           # Configuración central
6 |-- chatbot/
7 |   |-- llm.py            # Configuración LLM (Ollama/OpenAI
   )
8 |   |-- rag_chain.py       # Pipeline RAG + condensación de
   preguntas
9 |-- db/
10 |   |-- vector_store.py   # Wrapper ChromaDB (upsert
   idempotente)
11 |-- ontology/
12 |   |-- build_ontology.py  # Construcción del grafo RDF/OWL

```

```

13 |   |-- fib_ontology.py      # API de consulta (matching,
    |   SPARQL)
14 |   |-- fib_ontology.ttl    # Serializacion Turtle (generada)
15 | |-- retrieval/
16 |   |-- base.py            # Interfaz comun de resultados
17 |   |-- bm25.py            # Baseline lexico
18 |   |-- dense.py           # Baseline vectorial
19 |   |-- ontology_expansion.py # Expansion ontologica naive
20 |   |-- entity_linker.py    # Vinculacion de entidades
21 |   |-- controlled_ontology.py # Busqueda controlada por
    |   ontologia
22 |   |-- hybrid_rrf.py       # Fusion hibrida RRF
23 | |-- evaluation/
24 |   |-- queries.json        # Golden set (40 consultas, dev/
    |   test)
25 |   |-- evaluate.py         # Evaluacion automatica (Hit@k,
    |   MRR, nDCG)
26 |   |-- results/           # Resultados con timestamp (CSV +
    |   JSON)
27 | |-- processing/
28 |   |-- embedder.py         # Generacion embeddings
29 |   |-- pdf_loader.py       # Carga y chunking PDFs
30 | |-- scraped_data/
31 |   |-- fib_documents.json  # Datos scrapeados
32 | |-- chroma_db/           # Base de datos vectorial

```

7.3. Flujo de datos

7.3.1. Pipeline Offline (Indexación)

1. El **Scraper** recorre `fib.upc.edu` y extrae documentos estructurados (1.812 documentos de 411 páginas en la última ejecución).
2. El **constructor de la ontología** genera el grafo RDF a partir del esquema declarado y de los datos scrapeados (nombres oficiales de asignaturas, URLs canónicas).
3. El script de **ingesta** deduplica los documentos (priorizando URLs canónicas), los divide en chunks respetando párrafos y frases, y filtra fragmentos de bajo valor.
4. El **Embedder** genera vectores de 384 dimensiones para cada chunk (incluyendo título y sección como contexto).
5. **ChromaDB** almacena vectores, textos y metadatos de forma persistente e idempotente (*upsert*).

7.3.2. Pipeline Online (Consulta)

1. El usuario escribe una pregunta en la interfaz.

2. Si hay historial de conversación, el LLM **condensa** la pregunta de seguimiento en una pregunta autónoma.
3. El **retriever del modo activo** recupera los documentos más relevantes (el detalle de cada estrategia se da en el Capítulo 8).
4. El **LLM** genera una respuesta basada en los documentos, con el contexto ontológico como conocimiento estructurado adicional y con las fuentes citadas.
5. La **UI** muestra la respuesta, las fuentes y los detalles técnicos (query enriquecida, entidades detectadas, boosts aplicados).

Capítulo 8

Implementación

Este capítulo es el núcleo de la memoria y está contado, deliberadamente, en el orden en que se desarrolló el sistema. Me parece más honesto y más útil que presentar el resultado final como si hubiera nacido completo: muchas de las piezas que acabaron siendo centrales (la deduplicación canónica, la vinculación de entidades, el re-ranqueo controlado) no estaban en el diseño inicial, y entender *por qué* aparecieron explica mejor el sistema que cualquier descripción estática.

8.1. Adquisición de datos: Web Scraping

8.1.1. Punto de partida

El primer sprint se dedicó íntegramente a la adquisición de datos. La premisa del proyecto es que el sistema trabaja **únicamente** con información oficial de la FIB, así que todo empieza por extraer el contenido de `fib.upc.edu` y sus subpáginas.

El scraper implementa un recorrido BFS (*breadth-first search*) a partir de la página principal, con las siguientes reglas:

- Solo se siguen enlaces internos del dominio `fib.upc.edu`.
- Se excluyen recursos no textuales (imágenes, CSS, JS) y páginas de login.
- Se respeta un retardo entre peticiones para no sobrecargar el servidor.
- De cada página se extraen: URL, título, secciones (encabezados `h1-h4`) y el texto limpio de cada sección.

La extracción por secciones, en lugar de páginas enteras, fue una decisión temprana que resultó acertada: las páginas de la FIB suelen agrupar varios temas bajo encabezados distintos (por ejemplo, la página de matrícula tiene secciones de períodos, procedimiento y documentación), y tratar cada sección como documento independiente mejora la granularidad de la recuperación.

La última ejecución completa del scraper procesó **411 páginas** y produjo **1.812 documentos** (secciones), que tras la limpieza y deduplicación de la Sección 8.3 se convierten en unos **2.700 chunks** indexados.

8.1.2. Problemas encontrados

No todo fue tan directo como suena. Tres problemas concretos del scraping acabaron teniendo consecuencias en fases posteriores:

1. **Contenido duplicado entre URLs distintas.** La web de la FIB publica la misma página bajo varias rutas (por idioma, por titulación, por redirecciones históricas). Por ejemplo, la página de “places lliures” es idéntica para varios grados pero vive en URLs diferentes. Al principio no le di importancia; más adelante (Sección 8.3.1) resultó ser una fuente seria de problemas.
2. **Páginas dinámicas con poco texto.** Horarios, calendarios de exámenes y plazas libres generan su contenido útil con JavaScript o tablas, de modo que el HTML estático apenas contiene texto indexable. Son precisamente las páginas que más consultan los estudiantes.
3. **Mezcla de idiomas.** El corpus está mayoritariamente en catalán, pero hay secciones en castellano e inglés, y los usuarios preguntan en cualquiera de los dos primeros. Esto condicionó la elección del modelo de embeddings multilingüe.

8.2. La ontología de dominio

8.2.1. Primera iteración: un diccionario de sinónimos

Mi primera aproximación a la “ontología” fue, siendo sincero, un diccionario de Python: un mapa de conceptos a listas de sinónimos (`{"matricula": ["matriculació", "inscripció", ...]}`) que servía para expandir las consultas. Funcionaba como prueba de concepto, pero tenía tres carencias que se hicieron evidentes en cuanto el proyecto creció:

1. **No era una ontología.** No había clases, ni jerarquía, ni relaciones tipadas: solo vocabulario plano. Difícilmente podía defender en la memoria que el sistema estaba “basado en ontologías”.
2. **No escalaba.** Añadir las ~100 asignaturas del GEI con sus códigos y URLs a mano era inviable y frágil.
3. **No era consultable.** Cualquier operación no prevista (“dame todos los conceptos relacionados con trámites”) requería escribir código nuevo en lugar de una consulta declarativa.

8.2.2. Segunda iteración: grafo formal RDF/OWL

Después de investigar las opciones del ecosistema de la web semántica (Sección 2.1), rediseñé la ontología como un **grafo RDF formal** construido con `rdflib`, con esquema OWL y vocabulario SKOS, serializado en Turtle (`fib_ontology.ttl`). La Figura 8.1 muestra la arquitectura en dos capas.

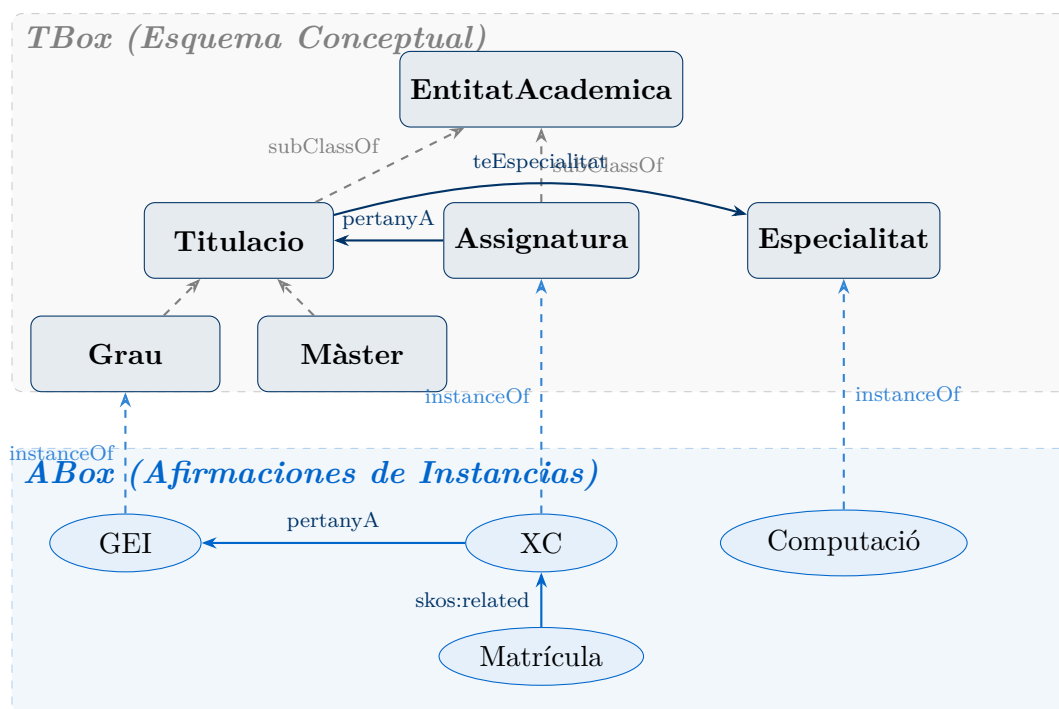


Figura 8.1: Arquitectura de la ontología en dos capas: distinción formal entre el esquema conceptual (TBox) y la población de instancias (ABox).

El esquema (TBox) define **7 clases OWL**:

- **EntitatAcademica**: clase raíz de todas las entidades del dominio.
- **Titulacio**, con subclases **Grau** y **Master**.
- **Assignatura**: asignaturas con código oficial.
- **Especialitat**: especialidades del GEI.
- **ConcepteAcademic**: conceptos transversales (matrícula, horarios, becas, normativa, trámites...).

3 propiedades de objeto relacionan instancias entre sí (**pertanyA**, **teEspecialitat**, **dinsEspecialitat**), y **4 propiedades de datos** conectan el grafo con el mundo operativo del buscador. Esta última decisión de diseño es probablemente la más importante de la ontología y merece la Tabla 8.1: la ontología no es un adorno conceptual, sino que cada propiedad tiene un **rol ejecutable** en el pipeline de recuperación.

Cuadro 8.1: Propiedades de datos y su rol operativo en el *retrieval*

Propiedad	Descripción ontológica	Rol en el pipeline
<code>recursCanonic</code>	URL oficial y definitiva del recurso en la web de la FIB.	Inyección de candidatos: garantiza que la página clave entre en el <i>pool</i> aunque su similitud semántica sea baja.
<code>pesIntencio</code>	Valor numérico [0, 1] que gradúa la especificidad del concepto.	Magnitud del <i>boost</i> en el reranqueo.
<code>patroUrl</code>	Fragmento de URL (<i>slug</i>) asociado al concepto.	<i>Matches</i> parciales durante el reranqueo para priorizar páginas hijas.
<code>codi</code>	Acrónimo o sigla oficial (ej. XC, GEI).	Activador del <i>entity linking</i> : si la consulta contiene el código, se dispara la inyección.

Para el vocabulario multilingüe se adoptó SKOS: cada instancia tiene una `skos:prefLabel` (etiqueta preferente) y tantas `skos:altLabel` como variantes léxicas se le conocen, con etiquetas de idioma (`@ca`, `@es`). Las relaciones asociativas entre conceptos (matrícula ↔ plazas libres ↔ calendario académico) se expresan con `skos:related`, y son las que alimentan la expansión de consultas mediante SPARQL.

8.2.3. Población semiautomática

La carencia de escalabilidad de la primera iteración se resolvió con un constructor (`build_ontology.py`) que puebla el ABox combinando dos fuentes:

1. **Esquema declarado a mano:** las clases, propiedades, titulaciones, especialidades y los ~19 conceptos académicos transversales con su vocabulario. Esta parte es curación manual y es donde reside el conocimiento del dominio.
2. **Instancias extraídas de los datos scrapeados:** las ~94 asignaturas del GEI se generan automáticamente cruzando los códigos oficiales con los títulos de sus páginas en `scraped_data`, de donde se extrae el nombre completo oficial de cada asignatura y su URL canónica.

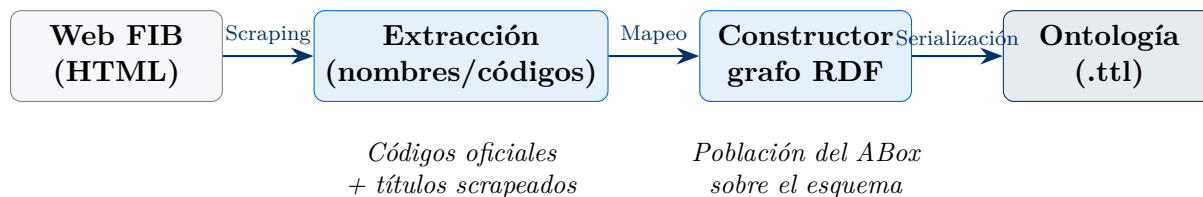


Figura 8.2: Pipeline de población semiautomática: la ontología se reconstruye desde los datos reales de la facultad, de modo que un cambio en la web se incorpora re-ejecutando el constructor.

El resultado es un grafo de **más de 1.000 tripletas** con **126 instancias**: 4 grados, 4 másters, 5 especialidades, 94 asignaturas y 19 conceptos académicos. La Figura 8.3 muestra una visualización del grafo completo generada a partir del fichero Turtle.

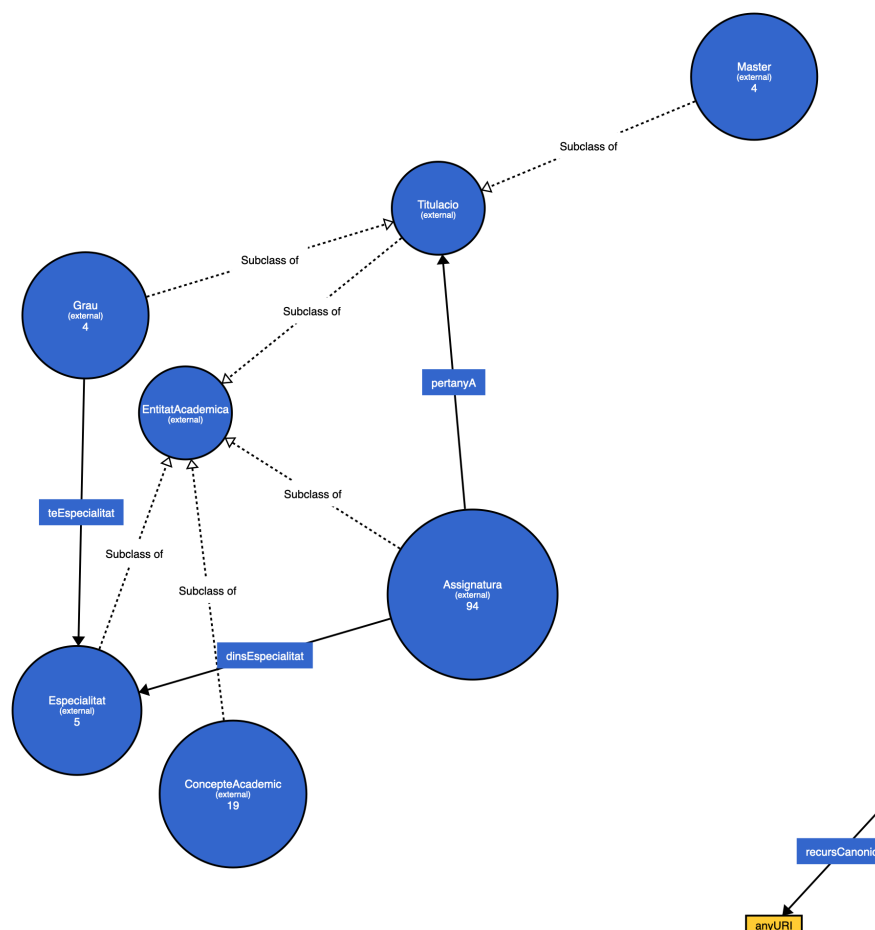


Figura 8.3: Visualización del grafo RDF de la ontología de la FIB. En el centro, las clases del esquema; alrededor, las instancias de asignaturas, titulaciones y conceptos con sus relaciones.

8.2.4. API de consulta

Sobre el grafo se construyó una capa de acceso (`fib_ontology.py`) que carga el TTL en memoria y ofrece las operaciones que el resto del sistema necesita: detección de conceptos en una consulta (con normalización de acentos y límites de palabra), expansión de términos vía `skos:related`, recuperación de recursos canónicos y pesos, índices de códigos de asignatura y siglas de titulación, y generación del contexto ontológico que se pasa al LLM. La expansión usa SPARQL directamente sobre el grafo:

Listing 8.1: Consulta SPARQL de expansión: términos relacionados de un concepto detectado

```

1 PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
2 PREFIX fib: <http://www.fib.upc.edu/ontology#>
3
4 SELECT ?relatedLabel ?canonicalUrl ?weight
5 WHERE {
6     ?concept    skos:prefLabel    ?label ;
7                skos:related      ?related .
8     ?related    skos:prefLabel    ?relatedLabel ;
9                fib:recursCanonic  ?canonicalUrl ;
10               fib:pesIntencio    ?weight .
11 }

```

Un ejemplo de ejecución completo, que además anticipa el mecanismo de re-ranqueo de la Sección 8.4.5:

1. **Consulta original:** “*com m’apunto a les assignatures?*”
2. **Detección:** “m’apunto” coincide con una `skos:altLabel` del concepto `Matrícula`.
3. **Expansión:** vía `skos:related` se añaden “*places lliures*” y “*calendari acadèmic*”.
4. **Recursos:** se recupera la URL canónica de matrícula y su `pesIntencio` (0.20).
5. **Resultado:** la página oficial de matrícula entra garantizada en el *pool* de candidatos y recibe un *boost* en el re-ranqueo, aunque el fraseo informal del estudiante no comparta ni una palabra con el texto de la página.

8.3. Indexación: del HTML a ChromaDB

La ingesta (`ingest.py`) transforma los documentos scrapeados en chunks indexados. Esta parte sufrió más reescrituras de las que me gustaría admitir, porque cada problema de calidad de datos que aparecía aguas arriba acababa manifestándose como un fallo de recuperación aguas abajo, difícil de diagnosticar.

8.3.1. Deduplicación con prioridad canónica

El primer problema serio: al evaluar las primeras versiones, había consultas que fallaban de forma aparentemente aleatoria. Investigando, descubrí que la causa estaba en la deduplicación. Muchas páginas de la FIB tienen **contenido idéntico bajo URLs distintas** (la misma página de plazas libres servida para varios grados). Mi deduplicación inicial, por hash de contenido, se quedaba con una URL *arbitraria* de cada grupo de duplicados. El resultado: a veces la URL canónica que la ontología esperaba no estaba indexada, y todo el mecanismo de inyección y boosts fallaba silenciosamente.

La solución fue hacer la deduplicación **consciente de la ontología**: cuando varios documentos comparten contenido, se conserva como representante la versión cuya URL es canónica según el grafo (`recursCanonic`), y el resto de URLs se guardan como metadato `duplicate_urls`. Es un detalle pequeño, pero ilustra bien una lección transversal del proyecto: **la ontología solo es útil si el índice y el grafo son coherentes entre sí**.

8.3.2. Chunking semántico

La primera versión troceaba el texto cada N caracteres, cortando frases por la mitad; los embeddings de esos fragmentos eran semánticamente confusos. La versión final usa un divisor recursivo (`RecursiveCharacterTextSplitter`) que intenta cortar por párrafos, luego por frases y solo en último caso por palabras, con chunks de ~ 1.000 caracteres y solapamiento de 150. Además, el texto que se vectoriza incluye el **título de la página y la sección** como prefijo, lo que da contexto al embedding (un chunk que dice “el período s’obre al juliol” es ambiguo; precedido de “Matrícula > Períodes” deja de serlo).

8.3.3. Filtrado de fragmentos de bajo valor y páginas de aterrizaje

El scraping produce inevitablemente ruido: migas de pan, menús, avisos de cookies, fragmentos residuales de pocas palabras. Se añadió un filtro que descarta chunks de menos de **120 caracteres** (`MIN_CHUNK_LENGTH`) o con menos de un 50 % de contenido alfabético.

Pero este filtro, tal cual, creaba un problema con las **páginas de aterrizaje críticas**: horarios, exámenes, profesorado o plazas libres tienen poquísimo texto estático (su contenido es dinámico), y el filtro las eliminaba del índice... justo las páginas que más se consultan. La solución fue una regla de excepción: **las páginas cuya URL es un recurso canónico de la ontología se indexan siempre**, con un umbral mínimo reducido (30 caracteres). De nuevo, la ontología actuando como fuente de verdad sobre qué es importante, independientemente del volumen de texto.

8.3.4. Identificadores robustos e idempotencia

Un último problema de ingeniería que vale la pena documentar porque costó varias horas de depuración: los identificadores de chunk originales (URL + primeros 200 caracteres) colisionaban cuando dos chunks empezaban igual, y ChromaDB rechazaba **lotes enteros** de 32 chunks por una sola colisión, perdiendo documentos de forma silenciosa. La versión final usa IDs deterministas (hash de URL + índice de chunk + hash del texto) y operaciones *upsert*, de modo que re-ejecutar la ingesta es idempotente y nunca pierde datos.

8.4. Las cinco estrategias de recuperación

Con el índice estable, el trabajo se centró en la pregunta central del TFG: ¿cómo recuperar mejor? Para poder responderla con rigor, todas las estrategias se implementaron bajo una **interfaz común** (`retrieval/base.py`): cada retriever recibe una consulta y devuelve una lista de resultados homogéneos (documento, metadatos, score, fuente de recuperación, boosts aplicados). Esto permite conmutarlas desde la UI y evaluarlas con el mismo script.

8.4.1. Baseline 1: BM25 (léxico)

El baseline léxico (`retrieval/bm25.py`) carga todos los chunks del índice, los tokeniza con normalización de acentos y *stopwords* de catalán y castellano, y construye un índice Okapi BM25 en memoria (Sección 2.3). Su perfil es el esperado: excelente cuando la consulta comparte términos exactos con el documento (códigos, nombres propios), inútil ante sinonimia o cambios de idioma.

8.4.2. Baseline 2: búsqueda vectorial pura (dense)

El baseline vectorial (`retrieval/dense.py`) es el RAG clásico: embedding de la consulta con MiniLM y búsqueda por similitud coseno en ChromaDB. Era el comportamiento de la primera versión del sistema completo, y conviene subrayarlo: **todo lo que viene después se mide contra esto**.

8.4.3. Estrategia 3: expansión ontológica naïve

La primera idea “semántica” fue la más intuitiva: detectar conceptos de la ontología en la consulta y añadir sus términos relacionados antes de la búsqueda vectorial (`retrieval/ontology_expansion.py`). Por ejemplo, “*quan puc matricular-me?*” se convierte en “*quan puc matricular-me? matrícula places lliures calendari acadèmic*”.

Mi expectativa era que esto mejoraría claramente al baseline. El resultado real (anticipo de la Tabla 9.1): mejora marginal, casi ruido. El análisis de errores fue revelador: añadir términos al texto de la consulta **diluye el embedding** —el vector resultante es un promedio semántico de más cosas— y solo ayuda cuando la consulta original era demasiado escueta. Este resultado negativo fue el punto de inflexión del proyecto: la ontología no aporta valor *reescribiendo la consulta*, sino *decidiendo qué documentos importan*. Eso es exactamente lo que hace la estrategia siguiente.

8.4.4. Vinculación de entidades (Entity Linking)

Antes de la estrategia controlada hay que explicar su pieza más delicada. Durante el análisis de errores apareció un patrón recurrente: las consultas con **siglas o códigos** (“*temari de XC*”, “*què necessito per fer el TFG?*”, “*notes de tall del GEI*”) fallaban estrepitosamente en los dos baselines. Para BM25, “XC” es un token rarísimo que además aparece poco en la página de la propia asignatura; para el embedding, una sigla de dos letras apenas aporta señal semántica.

La solución (`retrieval/entity_linker.py`) explota los índices de códigos de la ontología: las ~94 asignaturas con su *codi* y las titulaciones con sus siglas. La detección usa expresiones regulares con **límites de palabra** (`\b`) sobre la consulta, con reglas de sensibilidad a mayúsculas que tuvieron que refinarse empíricamente:

- Los códigos en general se detectan *case-insensitive*, porque los estudiantes escriben “xc” o “prop” sin mayúsculas.
- Sin embargo, hay códigos que colisionan con palabras comunes del catalán/castellano: CAP, PES, PAR, MAI, BIO, SOA, IES, EDO, ROB. La consulta “*quina nota cal per entrar?*” no menciona la asignatura CAP, aunque contenga la palabra “cap”. Para este conjunto de **códigos ambiguos**, el emparejamiento exige mayúsculas exactas: “CAP” sí dispara la vinculación, “cap” no.

Esta lista de códigos ambiguos se construyó mediante análisis de falsos positivos sobre las consultas de desarrollo, y es un buen ejemplo del tipo de conocimiento de dominio —pequeño, aburrido, decisivo— que no aparece en ningún paper pero marca la diferencia entre un sistema que funciona y uno que no.

Cada entidad vinculada se resuelve a su **URL canónica** vía el grafo, y esa información se entrega al re-ranqueador.

8.4.5. Estrategia 4: búsqueda controlada por ontología

La estrategia principal del trabajo (`retrieval/controlled_ontology.py`) reorganiza el papel de la ontología según la lección de la Sección 8.4.3: en lugar de modificar la consulta, **controla el conjunto de candidatos y su orden**. El algoritmo tiene cuatro fases:

1. **Recuperación amplia.** Se recuperan hasta $\max(8k, 40)$ candidatos de ChromaDB, usando tanto la consulta original como la enriquecida. La idea es que el recall bruto deje de ser el cuello de botella: el trabajo fino se hace después, en el re-ranqueo.
2. **Inyección de páginas canónicas.** Para cada entidad vinculada y cada concepto detectado, se buscan en el índice los chunks de su URL canónica y se **inyectan** en el pool de candidatos si no estaban. Esto garantiza que la página oficial de XC está en el pool cuando alguien pregunta por XC, tenga la similitud que tenga.
3. **Score base honesto.** Los documentos inyectados necesitan un score comparable al de los recuperados. Mi primera implementación les asignaba un valor fijo, pero esto distorsionaba el ranking de formas difíciles de razonar (a veces demasiado alto, a veces insuficiente). La versión final calcula la **similitud coseno real** entre el embedding de la consulta y el embedding almacenado de cada chunk inyectado: así todos los candidatos compiten en la misma escala y los boosts posteriores tienen una semántica clara (“cuánto confío en la señal ontológica por encima de la semántica”).
4. **Re-ranqueo guiado por el grafo.** Sobre el score base se aplican boosts aditivos cuyos valores y patrones **proviene de la ontología**, no de reglas codificadas:
 - **Entidad vinculada** (+0.45): el chunk pertenece a la URL canónica de una entidad detectada en la consulta. Es el boost más alto porque la señal es casi inequívoca.
 - **Concepto detectado** (+`pesIntencio`): el chunk pertenece a la URL canónica de un concepto detectado; la magnitud la decide el grafo (entre 0.12 y 0.30 según la especificidad del concepto).
 - **Coincidencia de slug** (+`pesIntencio` × 0.5): la URL del chunk contiene el `patroUrl` del concepto sin ser la página canónica (páginas hijas).
 - **Ámbito de titulación** (+0.15): si la consulta menciona una titulación (“del GEI”), los chunks de su subárbol de URLs reciben un impulso, lo que desambigua entre las páginas equivalentes de grados distintos.

Quiero subrayar la diferencia entre esto y un sistema de reglas *hardcodeadas*: aquí el código solo define los *mecanismos* (inyección, tipos de boost), mientras que el *conocimiento* (qué URL es canónica de qué, con qué peso, con qué patrones) vive en el grafo RDF y se consulta en tiempo de ejecución. Añadir un concepto nuevo no requiere tocar el código: basta añadirlo a la ontología y reconstruir el TTL (requisito RNF05).

8.4.6. Estrategia 5: fusión híbrida (RRF)

La última estrategia (`retrieval/hybrid_rrf.py`) fusiona los rankings de dense, BM25 y controlada mediante *Reciprocal Rank Fusion* (Sección 2.6), con $k = 60$ y pesos 1.0/0.6/0.6 (controlada/BM25/dense). La fusión se hace a nivel de URL —no de chunk— para que varios fragmentos de la misma página no compitan entre sí.

La motivación teórica es sólida (señales complementarias), y de hecho la literatura suele recomendar híbridos por defecto. Como se verá en la evaluación, en nuestro dominio el resultado fue otro, y entender por qué es uno de los análisis más interesantes del Capítulo 9.

8.5. Pipeline RAG y generación

8.5.1. Condensación de preguntas de seguimiento

Un problema clásico de los chatbots RAG que apareció en cuanto empecé a probar conversaciones reales: la segunda pregunta del usuario (“*i quan obre el període?*”) es irresoluble sin el contexto de la primera (“*com funciona la matrícula?*”). La solución estándar, que adopté, es la **condensación**: antes de recuperar, un LLM reescribe la pregunta de seguimiento como pregunta autónoma usando el historial (“*quan obre el període de matrícula?*”), y es esa pregunta condensada la que entra al retriever. El coste es una llamada extra al LLM solo cuando hay historial.

8.5.2. Prompt y fundamentación

El prompt del sistema instruye al modelo para responder **exclusivamente** con la información de los documentos recuperados, en el idioma del usuario, citando las fuentes, y declarando explícitamente cuando la información no está disponible. Además de los documentos, el prompt incluye el **contexto ontológico**: los conceptos detectados, sus relaciones y sus recursos canónicos, en formato estructurado. En las pruebas cualitativas esto redujo visiblemente las respuestas evasivas en preguntas sobre entidades (el modelo “sabe” que XC es Xarxes de Computadors y pertenece al GEI aunque el chunk recuperado no lo diga).

8.6. Interfaz de usuario

La interfaz (`app.py`, Streamlit) evolucionó de un chat simple a una herramienta de demostración y análisis del sistema completo:

- **Chat conversacional** con historial, fuentes citadas y streaming de la respuesta.

- **Selector de estrategia** en la barra lateral: las cinco estrategias son conmutables en vivo, lo que convierte la UI en una demo directa del experimento central del TFG.
- **Detalles técnicos por respuesta**: un panel expandible muestra el flujo completo de la consulta (original → condensada → enriquecida), las entidades vinculadas, y cada documento recuperado con su score, similitud y los boosts aplicados como etiquetas.
- **Explorador de la ontología**: permite navegar el grafo por clase e instancia, viendo etiquetas SKOS, relaciones, recursos canónicos y pesos. Cumple una función didáctica: hacer visible el conocimiento que guía la búsqueda.
- **Comparador de estrategias**: dada una consulta, la ejecuta contra los cinco modos en paralelo y muestra el top-3 de URLs de cada uno en una tabla. Fue una herramienta de análisis de errores durante el desarrollo que acabó integrada en la UI.

Los componentes pesados (índice vectorial, grafo de la ontología, cadena RAG) se cachean como recursos de Streamlit, de modo que el cambio de estrategia es instantáneo y la única latencia perceptible es la generación del LLM.

Capítulo 9

Evaluación y Resultados

9.1. Diseño de la evaluación

9.1.1. Del análisis cualitativo a la evaluación formal

La primera versión del sistema se “evaluaba” de manera informal: probaba consultas a mano y miraba si la respuesta parecía razonable. Esto sirve para desarrollar, pero no para sostener ninguna afirmación de la memoria. La evaluación definitiva se diseñó con tres principios:

1. **Automática y reproducible:** un script (`evaluation/evaluate.py`) ejecuta todas las estrategias contra el mismo banco de consultas y produce los resultados en CSV/JSON con marca de tiempo. Cualquier cifra de este capítulo se puede regenerar con un comando.
2. **Centrada en la recuperación:** se evalúa qué documentos devuelve cada estrategia, no el texto generado por el LLM. La razón es metodológica: la generación es idéntica en todos los modos, de manera que cualquier diferencia de calidad proviene de la recuperación; y evaluar texto generado requeriría jueces humanos o LLM-as-judge, con la variabilidad que conllevan.
3. **Con control del sobreajuste:** las reglas y el vocabulario de la ontología se refinaron mediante análisis de errores, y eso crea un riesgo real de ajustarse al banco de pruebas. La mitigación se explica a continuación.

9.1.2. El banco de consultas (golden set)

Se construyó un conjunto de **40 consultas** (`evaluation/queries.json`) representativas de lo que pregunta un estudiante real: matrícula y trámites, asignaturas por código y por nombre, notas de corte, horarios y exámenes, becas, movilidad, prácticas, TFG, servicios... Las consultas están escritas en catalán y castellano, con fraseos coloquiales (“*com m’apunto a les assignatures?*”) además de formales, porque el desalineamiento de vocabulario es precisamente el fenómeno que queremos medir. Cada consulta tiene anotadas sus **URLs relevantes**, verificadas manualmente contra la web de la FIB.

La decisión más importante del diseño es la **división dev/test**: 20 consultas de desarrollo (*dev*) que sí utilicé para analizar errores y refinar el vocabulario de la ontología,

y **20 consultas de test (held-out) que no se miraron durante el desarrollo**. La comparación entre ambos conjuntos permite detectar sobreajuste: si una estrategia funciona en dev pero se hunde en test, sus reglas están memorizando el banco de pruebas en lugar de generalizar. Esta precaución resultó muy informativa, como se verá.

9.1.3. Métricas

Se emplean las métricas estándar de recuperación de información, calculadas sobre las URLs únicas del top-5 de cada estrategia:

- **Hit@k** ($k = 1, 3, 5$): proporción de consultas con al menos una URL relevante en el top- k . Hit@1 mide precisión estricta; Hit@5, cobertura práctica (en la UI se muestran 5 fuentes).
- **MRR** (*Mean Reciprocal Rank*): media de $1/\text{rank}$ de la primera URL relevante. Premia que lo correcto esté arriba.

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (9.1)$$

- **P@5 / R@5**: precisión y exhaustividad clásicas en el top-5.
- **nDCG@5**: ganancia acumulada descontada normalizada, que pondera la relevancia por la posición con descuento logarítmico.

9.2. Resultados de la recuperación

La Tabla 9.1 resume los resultados globales de las cinco estrategias sobre las 40 consultas.

Cuadro 9.1: Resultados globales de las cinco estrategias de recuperación (40 consultas)

Estrategia	Hit@1	Hit@3	Hit@5	MRR	nDCG@5
BM25 (léxico)	20,0 %	47,5 %	55,0 %	0,337	0,347
Dense (vectorial)	15,0 %	30,0 %	37,5 %	0,233	0,245
Expansión ontológica	10,0 %	30,0 %	47,5 %	0,234	0,264
Controlada (ontología)	75,0 %	87,5 %	92,5 %	0,820	0,813
Híbrida (RRF)	42,5 %	62,5 %	70,0 %	0,525	0,532

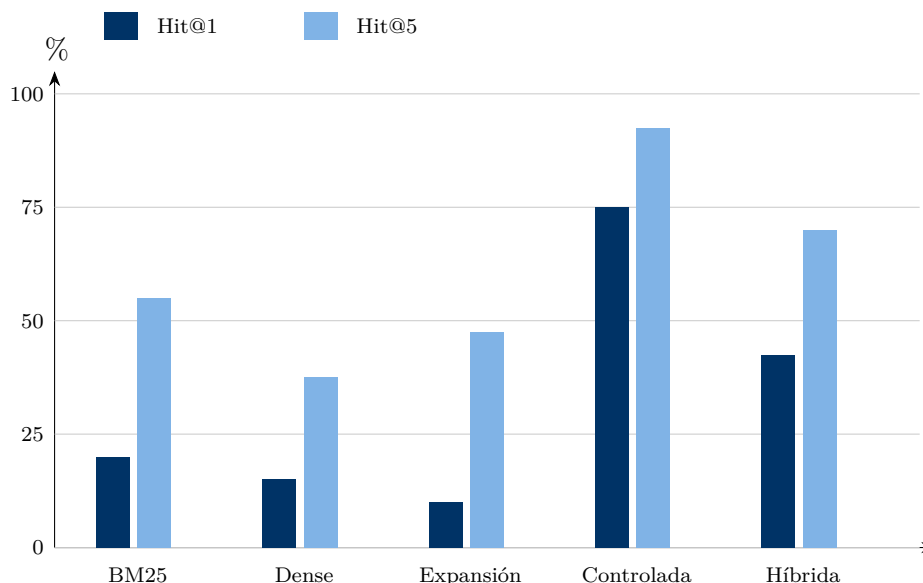


Figura 9.1: Hit@1 y Hit@5 de las cinco estrategias sobre el conjunto completo de 40 consultas.

Los resultados responden directamente a las tres preguntas de investigación del Capítulo 1:

P1 — ¿Sirve la expansión de consulta por sí sola? Apenas. La expansión ontológica naive mejora ligeramente el Hit@5 del baseline vectorial (47,5 % vs. 37,5 %) pero *empeora* su Hit@1 (10 % vs. 15 %): los términos añadidos amplifican el recall a costa de diluir la precisión del embedding. Es un resultado negativo, coherente con la experiencia histórica de la expansión de consultas en IR [9], y para mí fue la lección más formativa del proyecto.

P2 — ¿Qué aporta la integración profunda? La diferencia entre usar la ontología como diccionario (expansión) y usarla como **fuentes de conocimiento canónico** (estrategia controlada) es drástica: de 47,5 % a **92,5 % de Hit@5**, y un MRR que pasa de 0,23 a **0,82**. La estrategia controlada multiplica por 2,5 el Hit@5 del mejor baseline y por 3,5 su Hit@1. El análisis por categorías muestra que la mejora se concentra exactamente donde se diseñó: consultas con siglas y códigos (donde el entity linking inyecta la página correcta) y consultas sobre páginas de aterrizaje de poco texto (donde la inyección canónica compensa la falta de señal semántica).

P3 — ¿Híbrido o estrategia dominante? Contra mi intuición inicial, el híbrido RRF (70 % Hit@5) quedó claramente por debajo de la estrategia controlada sola. La Sección 9.4 analiza el porqué.

9.2.1. Generalización: dev vs. test

La Tabla 9.2 desglosa los resultados por partición, que es donde se comprueba si las reglas generalizan o memorizan.

Cuadro 9.2: Desglose por partición: 20 consultas de desarrollo vs. 20 consultas held-out

Estrategia	Split	Hit@1	Hit@5	MRR	nDCG@5
BM25	dev	20,0 %	70,0 %	0,406	0,461
	test	20,0 %	40,0 %	0,268	0,233
Dense	dev	10,0 %	35,0 %	0,189	0,229
	test	20,0 %	40,0 %	0,277	0,261
Expansión	dev	10,0 %	45,0 %	0,238	0,284
	test	10,0 %	50,0 %	0,231	0,244
Controlada	dev	80,0 %	95,0 %	0,867	0,869
	test	70,0 %	90,0 %	0,773	0,758
Híbrida	dev	50,0 %	75,0 %	0,602	0,626
	test	35,0 %	65,0 %	0,448	0,438

La lectura clave: la estrategia controlada cae solo 5 puntos de Hit@5 entre dev y test (95 % $\rightarrow$ 90 %), una degradación moderada que indica que **los mecanismos generalizan** a consultas no vistas. El conocimiento que explota (URLs canónicas, códigos, relaciones) es estructural del dominio, no específico de las consultas con las que se desarrolló. En contraste, BM25 se desploma de 70 % a 40 %: su buen resultado en dev dependía de coincidencias léxicas afortunadas que no se repiten en test.

Esta tabla es también una declaración metodológica: habría sido fácil reportar solo los resultados de dev (95 % de Hit@5 queda mejor que 92,5 %), pero el valor de la evaluación está precisamente en demostrar que el sistema no está sobreajustado.

9.3. Impacto de la ontología: análisis por mecanismo

Dado que la estrategia controlada combina varios mecanismos, conviene atribuir la mejora. Del análisis de los resultados detallados (archivo `details` de la evaluación) se extraen tres patrones:

1. **El entity linking es decisivo en las consultas sobre asignaturas** (5 de las 40, además de las que mencionan siglas de titulación). En dense, este tipo de consulta falla casi siempre (la sigla no aporta señal); con vinculación + inyección, la página de la asignatura aparece en el top-1 de manera consistente.
2. **La inyección canónica rescata las páginas de aterrizaje.** Horarios, exámenes, plazas libres: páginas casi sin texto que ninguna estrategia de contenido puede encontrar. La combinación indexación-forzada (Sección 8.3.3) + inyección + boost las pone arriba cuando el concepto se detecta.
3. **El ámbito de titulación resuelve la ambigüedad entre grados.** Consultas como “places lliures del GEI” devolvían antes la página equivalente de otro grado (contenido casi idéntico); el boost de subárbol corrige la mayoría de estos casos.

Igual de informativo es lo que **no** funcionó: la expansión de términos, que era mi apuesta inicial, resultó ser el mecanismo menos valioso. Si tuviera que resumir el

hallazgo central del TFG en una frase: *en un dominio cerrado, la ontología aporta más como mapa de recursos canónicos que como diccionario de sinónimos.*

9.4. Evaluación de la estrategia híbrida (RRF)

El resultado del híbrido merece sección propia porque contradice la recomendación habitual de la literatura (“cuando dudes, fusiona”). Con 70 % de Hit@5 y MRR 0,525, el RRF quedó 22,5 puntos por debajo de la estrategia controlada que es uno de sus componentes. ¿Cómo puede una fusión empeorar a su mejor miembro?

La explicación está en la aritmética del RRF. La fusión por rango asume que los sistemas combinados tienen **calidades comparables**; cuando uno es claramente superior (controlada: 0,82 MRR) y los otros notablemente inferiores (BM25: 0,34; dense: 0,23), cada documento irrelevante que BM25 o dense colocan en sus primeras posiciones recibe una puntuación RRF alta y **desplaza hacia abajo los documentos correctos** de la estrategia controlada. El ruido de los débiles diluye la precisión del fuerte: en consultas de matrícula o calendario, BM25 aporta documentos lexicalmente relacionados (menciones de “matrícula” en noticias, normativas antiguas) que no son las páginas canónicas.

Lo intenté mitigar bajando los pesos de los componentes débiles (1.0/0.6/0.6), lo que mejoró algo el resultado, pero la conclusión empírica se mantiene: **en este dominio, la fusión solo tendría sentido si los componentes fueran más equilibrados**. Decidí documentar el resultado negativo en lugar de esconderlo, porque responde limpiamente a P3 y porque el análisis del fallo (dilución por desequilibrio de calidad) es generalizable a otros sistemas RAG.

Una reflexión adicional: la estrategia controlada ya es, internamente, un híbrido — combina similitud semántica con conocimiento estructurado — pero con una integración *jerárquica* (la semántica propone, la ontología corrige) en lugar de *democrática* (todos votan). Los resultados sugieren que en dominios cerrados con conocimiento curado, la integración jerárquica es preferible.

9.5. Evaluación cualitativa del sistema completo

Además de las métricas de recuperación, se realizaron pruebas cualitativas del pipeline completo (recuperación + generación) con conversaciones reales. Algunas observaciones:

- Las respuestas generadas son fieles a los documentos: el prompt restrictivo y el contexto ontológico hacen que el modelo decline responder cuando no tiene la información, en lugar de inventar.
- La condensación de preguntas funciona bien en seguimientos directos (“i quan obre?”) y razonablemente en cambios de tema parciales.
- La latencia de generación con Llama 3.2 (3B) en CPU es perceptible (varios segundos hasta el primer token), aunque la recuperación en sí se mantiene por debajo de los 2 segundos del requisito RNF01.

9.6. Evaluación con usuarios (SUS): diseño del estudio

Para complementar la evaluación cuantitativa de la recuperación con una medida de la experiencia de uso, se ha diseñado un estudio de usabilidad basado en el **System Usability Scale** (SUS) [10], el cuestionario estándar de 10 ítems con escala Likert de 5 puntos, cuya puntuación agregada (0–100) permite situar el sistema en percentiles bien estudiados (la media de la industria se sitúa en 68; por encima de 80 se considera excelente).

9.6.1. Metodología diseñada

- **Participantes:** 4–6 estudiantes de grado de la FIB, usuarios habituales de la web de la facultad, sin relación con el proyecto.
- **Tareas:** cada participante resuelve 4 tareas representativas usando únicamente SemanticFIB: (1) averiguar el período de matrícula, (2) encontrar el temario de una asignatura concreta (p.ej. Xarxes de Computadors), (3) consultar el calendario de exámenes, (4) comprobar las plazas libres de su grado.
- **Protocolo:** sin ayuda del facilitador; se registra si la tarea se completa y el tiempo aproximado. Al finalizar, el participante rellena el cuestionario SUS y se recoge feedback libre.
- **Análisis:** puntuación SUS individual y media, tasa de éxito por tarea, y síntesis del feedback cualitativo.

En el momento de redactar esta memoria el estudio está diseñado y pendiente de ejecución con participantes reales; los resultados se incorporarán en la versión final del documento. Las pruebas informales realizadas durante el desarrollo con compañeros apuntan a dos líneas de mejora ya identificadas: la latencia del primer token cuando el LLM se ejecuta en CPU, y la conveniencia de poder exportar la conversación.

9.7. Limitaciones observadas

Un análisis honesto de los resultados debe incluir lo que el sistema no hace bien:

1. **Dependencia de la cobertura de la ontología.** La estrategia controlada solo despliega su ventaja cuando detecta algo (concepto, entidad, titulación). En las consultas de test que fallaron, la causa dominante fue vocabulario no cubierto: fraseos que ninguna `altLabel` captura. Ampliar cobertura es barato (editar el constructor del grafo) pero manual.
2. **Contenido dinámico no indexable.** El sistema enlaza a las páginas de horarios o plazas, pero no puede responder “¿quedan plazas en XC ahora mismo?” porque ese dato vive en tablas dinámicas. Resolverlo requeriría integración con APIs internas.

3. **El corpus envejece.** El índice refleja la web en el momento del scraping; la actualización requiere re-ejecutar el pipeline offline (automatizable, pero no implementado como servicio).
4. **Generación limitada por el modelo local.** Llama 3.2 3B comete ocasionalmente errores de matiz en catalán y su latencia en CPU es alta. El diseño permite conmutar a modelos mayores, con el coste correspondiente.
5. **Evaluación de la generación pendiente.** Se evaluó la recuperación exhaustivamente, pero la calidad del texto generado solo cualitativamente; una evaluación formal (fidelidad, completitud) queda como trabajo futuro.

Capítulo 10

Planificación Temporal y Económica

10.1. Planificación temporal

El proyecto se ha ejecutado en 5 sprints a lo largo de un cuatrimestre. Respecto a la planificación inicial, hubo una desviación relevante que merece constar: el sprint 4 estaba previsto solo para “pipeline RAG e integración LLM”, pero el resultado negativo de la expansión ontológica (Sección 8.4.3) obligó a rediseñar la estrategia principal (vinculación de entidades + re-ranqueo controlado), absorbiendo horas previstas para la interfaz. La interfaz se simplificó apoyándose en componentes nativos de Streamlit, lo que permitió recuperar el tiempo sin recortar la evaluación.

Cuadro 10.1: Desglose temporal por sprints

Sprint	Duración	Tareas principales	Horas
1	3 sem.	Investigación, diseño inicial, setup	80
2	3 sem.	Scraping, procesamiento de datos, baselines	90
3	3 sem.	Ontología formal, indexación, ChromaDB	85
4	3 sem.	Entity linking, re-ranqueo, híbrido, RAG, UI	100
5	3 sem.	Golden set, evaluación, análisis, memoria	90
Total			445

10.2. Gestión económica

10.2.1. Costes de personal

Cuadro 10.2: Costes de personal

Rol	Horas	€/hora	Total
Jefe de proyecto	60	30	1.800€
Ingeniero de datos	120	25	3.000€
Desarrollador backend	150	22	3.300€
Desarrollador frontend	65	22	1.430€
Tester/Evaluador	50	18	900€
Total personal	445		10.430€

10.2.2. Costes de hardware y software

Cuadro 10.3: Costes genéricos (hardware y software)

Concepto	Coste
Amortización portátil (4 meses / 48 meses vida útil)	100€
Electricidad (4 meses)	40€
Conexión Internet (4 meses)	80€
Software (todo open-source/gratuito)	0€
Servicios cloud (no aplica — todo local)	0€
Total genéricos	220€

10.2.3. Presupuesto total

Cuadro 10.4: Presupuesto total del proyecto

Concepto	Coste
Personal	10.430€
Genéricos (hardware/infra)	220€
Contingencia (10 %)	1.065€
Imprevistos (5 %)	533€
TOTAL	12.248€

Nota: el coste real desembolsado en software y servicios es de 0€ gracias al uso exclusivo de herramientas open-source y modelos locales; la decisión de no usar APIs de pago (que habrían añadido un coste variable por token) fue tanto económica como de privacidad.

Capítulo 11

Sostenibilidad

11.1. Dimensión económica

El proyecto demuestra que es posible construir un sistema de búsqueda semántica avanzado con **coste cero en infraestructura**:

- LLM local (Ollama): sin suscripciones mensuales.
- ChromaDB embebido: sin servidores dedicados.
- Modelos y librerías open-source: sin licencias.
- Scraping propio: sin APIs de pago.

Esto hace el sistema sostenible a largo plazo y replicable por otras facultades: el único trabajo de adaptación sería el scraper y la población de la ontología con el vocabulario del nuevo dominio.

11.2. Dimensión ambiental

- **Consumo energético:** el uso de un modelo de 3B parámetros (frente a los cientos de miles de millones de los modelos comerciales) reduce drásticamente el consumo por inferencia, y el modelo de embeddings (120MB) es aún más ligero.
- **Computación local:** no genera tráfico de red innecesario ni depende de centros de datos remotos.
- **Reutilización:** el scraping se ejecuta una vez por actualización y el índice se reutiliza para todas las consultas; la ingesta idempotente evita reprocesados completos.

11.3. Dimensión social

- **Accesibilidad:** información disponible 24/7 en catalán y castellano.
- **Igualdad:** todos los estudiantes tienen el mismo acceso independientemente de su horario o de su familiaridad con la estructura de la web.

- **Transparencia:** las fuentes citadas y el panel de detalles técnicos permiten verificar (y auditar) cada respuesta.
- **Privacidad:** ningún dato de usuario sale del sistema local.

Capítulo 12

Conclusiones y Trabajo Futuro

12.1. Conclusiones

Este TFG partía de una hipótesis —que el conocimiento explícito del dominio puede mejorar la recuperación semántica en un dominio cerrado— y la ha confirmado, aunque no por el camino que esperaba al empezar. Las conclusiones principales son:

1. **La ontología mejora la recuperación de forma drástica, pero no como diccionario sino como mapa.** La expansión de consultas con sinónimos (la aplicación “obvia” de una ontología) apenas movió las métricas; en cambio, usar el grafo como fuente de conocimiento canónico —qué página es la oficial de cada concepto, qué código corresponde a qué asignatura— llevó el Hit@5 del 37,5 % al 92,5 % y el MRR de 0,23 a 0,82.
2. **Los resultados generalizan.** La diferencia de solo 5 puntos entre las consultas de desarrollo y las held-out (95 % vs. 90 % de Hit@5) indica que la mejora proviene de la estructura del dominio y no del sobreajuste a las consultas de prueba.
3. **La fusión híbrida no siempre ayuda.** El RRF de tres señales quedó claramente por debajo de la mejor estrategia individual: cuando un componente domina, la fusión por rango lo diluye. Documentar este resultado negativo es parte del valor experimental del trabajo.
4. **La calidad de los datos es condición previa.** Una parte sustancial del esfuerzo fue de ingeniería de datos (deduplicación canónica, chunking semántico, filtrado con excepciones, IDs robustos); ninguna estrategia de recuperación funciona sobre un índice incoherente.
5. **RAG fundamenta las respuestas.** Restringir el LLM a documentos reales, con contexto ontológico estructurado, produce respuestas verificables y reduce las alucinaciones a casos residuales.
6. **Es viable a coste cero.** Ollama + ChromaDB + Sentence Transformers + rdflib permiten un sistema completo, local y gratuito.

12.2. Objetivos alcanzados

Cuadro 12.1: Grado de cumplimiento de los objetivos

Objetivo	Estado
Scraper robusto para fib.upc.edu	Alcanzado
Ontología formal (RDF/OWL) con población semiautomática	Alcanzado
Cinco estrategias de recuperación comparables	Alcanzado
Pipeline RAG funcional con condensación conversacional	Alcanzado
Evaluación automática con golden set y split dev/test	Alcanzado
Interfaz web con comparador y explorador de la ontología	Alcanzado
Funcionamiento 100 % local con citación de fuentes	Alcanzado
Estudio de usabilidad con usuarios (SUS)	Diseñado, pendiente de ejecución

12.3. Trabajo futuro

1. **Ejecutar el estudio SUS** diseñado en la Sección 9.6 con estudiantes reales.
2. **Ampliar la cobertura de la ontología:** más conceptos, más variantes léxicas, y extender las asignaturas más allá del GEI (másters, GIA, GCED). El análisis de errores indica que es la vía de mejora con mejor relación esfuerzo/beneficio.
3. **Actualización automática:** programar scraping + reconstrucción de la ontología + re-ingesta periódicos.
4. **Evaluación de la generación:** medir formalmente fidelidad y completitud de las respuestas (jueces humanos o LLM-as-judge).
5. **Híbrido condicional:** en lugar de fusionar siempre, aprender a *cuándo* conviene cada estrategia (p.ej. usar BM25 solo si no se detecta ningún concepto), lo que podría recuperar el valor de las señales complementarias sin la dilución observada.
6. **Feedback de usuarios:** valoración de respuestas en la UI para mejora continua y detección de huecos de vocabulario.
7. **Despliegue e integración:** publicar el servicio para los estudiantes e integrar con el Racó para datos personalizados (expediente, horario propio).

Bibliografia

- [1] Gruber, T. R. (1993). *A Translation Approach to Portable Ontology Specifications*. Knowledge Acquisition, 5(2), 199–220.
- [2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS), 30.
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 33.
- [4] Reimers, N., Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP 2019.
- [5] Robertson, S., Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.
- [6] Cormack, G. V., Clarke, C. L. A., Büttcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. Proceedings of SIGIR 2009.
- [7] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130.
- [8] Baek, J., Aji, A. F., Saffari, A. (2023). *Knowledge-Augmented Language Model Prompting for Zero-Shot Knowledge Graph Question Answering*. Proceedings of NLRSE Workshop, ACL 2023.
- [9] Bhogal, J., Macfarlane, A., Smith, P. (2007). *A review of ontology based query expansion*. Information Processing & Management, 43(4), 866–886.
- [10] Brooke, J. (1996). *SUS: A ‘quick and dirty’ usability scale*. En: Usability Evaluation in Industry. Taylor & Francis.
- [11] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT.
- [12] W3C (2014). *RDF 1.1 Concepts and Abstract Syntax*. <https://www.w3.org/TR/rdf11-concepts/>

- [13] W3C (2009). *SKOS Simple Knowledge Organization System Reference*. <https://www.w3.org/TR/skos-reference/>
- [14] RDFLib Team (2024). *RDFLib: A Python library for working with RDF*. <https://rdflib.readthedocs.io/>
- [15] Chroma (2023). *Chroma: The open-source embedding database*. <https://www.trychroma.com/>
- [16] Ollama (2024). *Ollama: Get up and running with large language models locally*. <https://ollama.com/>
- [17] LangChain (2023). *LangChain: Building applications with LLMs through composability*. <https://www.langchain.com/>
- [18] Streamlit (2023). *Streamlit: A faster way to build and share data apps*. <https://streamlit.io/>
- [19] Richardson, L. (2023). *Beautiful Soup: A Python library for pulling data out of HTML and XML files*. <https://www.crummy.com/software/BeautifulSoup/>
- [20] Reimers, N., Gurevych, I. (2023). *Sentence Transformers: Multilingual Sentence, Paragraph, and Image Embeddings using BERT & Co*. <https://www.sbert.net/>
- [21] Meta AI (2024). *Llama 3.2: Lightweight, text-only models*. <https://ai.meta.com/llama/>

Apéndice A

Instrucciones de Instalación

A.1. Requisitos previos

- Python 3.9 o superior
- Ollama instalado con el modelo llama3.2
- 4GB de espacio en disco libre
- 8GB RAM mínimo (16GB recomendado)

A.2. Instalación y ejecución

Listing A.1: Comandos de instalación y ejecución

```
1 # 1. Clonar repositorio
2 git clone <repo-url>
3 cd semanticfib
4
5 # 2. Entorno virtual y dependencias
6 python3 -m venv venv
7 source venv/bin/activate
8 pip install -r requirements.txt
9
10 # 3. Instalar e iniciar Ollama
11 brew install ollama
12 ollama pull llama3.2
13
14 # 4. Scraping de datos
15 python3 scraper.py
16
17 # 5. Construir la ontologia (genera fib_ontology.ttl)
18 python3 -m ontology.build_ontology
19
20 # 6. Indexacion (dedup + chunking + embeddings)
21 python3 ingest.py --from-scrape --reset
22
23 # 7. Evaluacion de las 5 estrategias (opcional)
```

```
24 python3 -m evaluation.evaluate
25
26 # 8. Ejecutar la aplicacion
27 python3 -m streamlit run app.py
```

Apéndice B

Extracto de la Ontología (Turtle)

Fragmento representativo de `fib_ontology.ttl` mostrando una asignatura, un concepto académico y sus propiedades operativas:

Listing B.1: Extracto de `fib_ontology.ttl`

```
1 @prefix fib: <http://www.fib.upc.edu/ontology#> .
2 @prefix skos: <http://www.w3.org/2004/02/skos/core#> .
3 @prefix owl: <http://www.w3.org/2002/07/owl#> .
4
5 fib:Assignatura a owl:Class ;
6     rdfs:subClassOf fib:EntitatAcademica .
7
8 fib:assignatura_XC a fib:Assignatura ;
9     skos:prefLabel "Xarxes de Computadors"@ca ;
10    fib:codi "XC" ;
11    fib:pertanyA fib:grau_GEI ;
12    fib:recursCanonic
13        "https://www.fib.upc.edu/ca/estudis/graus/grau-en-
14            enginyeria-informatica/pla-destudis/assignatures/XC" .
15
16 fib:concepte_matricula a fib:ConcepteAcademic ;
17     skos:prefLabel "Matricula"@ca ;
18     skos:altLabel "matriculacio"@ca, "automatricula"@ca,
19         "inscripcio"@ca, "matricularse"@es ;
20     skos:related fib:concepte_places_lliures,
21         fib:concepte_calendari_academic ;
22     fib:recursCanonic "https://www.fib.upc.edu/ca/estudis/graus
23         /.../matricula" ;
24     fib:pesIntencio "0.20"^^xsd:decimal ;
25     fib:patroUrl "matricula" .
```

Apéndice C

Ejemplos de Consultas

Cuadro C.1: Ejemplos de consultas y comportamiento del sistema (estrategia controlada)

Consulta	Comportamiento
¿Cómo funciona la matrícula en el GEI?	Detecta <i>matrícula</i> + <i>GEI</i> ; inyecta la página canónica de matrícula y prioriza el subárbol del GEI. Responde con el proceso, fechas y enlaces.
Temari de XC	Vincula la entidad XC (Xarxes de Computadors); la página oficial de la asignatura aparece en el top-1.
¿Qué especialidades tiene el grado?	Detecta <i>especialitat</i> ; lista las 5 especialidades con sus enlaces.
Vull anar d'Erasmus	“Erasmus” es <code>altLabel</code> de <i>mobilitat</i> ; recupera la página oficial de movilidad y normativa asociada.
quina nota cal per entrar?	Detecta <i>notes de tall</i> (sin disparar el código ambiguo CAP); responde con las notas de corte y la fuente.

Apéndice D

Categorías del Banco de Consultas

Cuadro D.1: Distribución del golden set por categoría (40 consultas, 20 dev / 20 test; 37 en catalán, 3 en castellano)

Categoría	Núm. consultas
Asignaturas (código o nombre)	5
Matrícula	3
Movilidad / Erasmus	3
Exámenes y horarios	4
Profesorado y plazas libres	4
Becas, trámites y normativa	6
TFG y especialidades	4
Calendario académico	2
Prácticas y empresa	3
Grados, plan de estudios y acceso	4
Contacto e información general	2